



Project Report on

Multiplayer Narrative Driven Game using Memory Augmented LLM for Real-Time Social Deception

*Submitted in partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology

in

Computer Science and Engineering

By

Vishwanath Pradeep (U2203212)

Serah Grace Kurian (U2203192)

Nandhaakishore S (U2203149)

Sreenandan Pradeep (U2203199)

Under the guidance of

Mr. Sebin Jose

**Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)**

Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

*This is to certify that the Project Report entitled "Multiplayer Narrative Driven Game using Memory Augmented LLM for Real-Time Social Deception" is a bonafide record of the work done by **Vishwanath Pradeep (U2203212)**, **Serah Grace Kurian(U2203192)**, **Nandhaakishore S (U2203149)**, and **Sreenandan Pradeep (U2203199)**, submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in "Computer Science and Engineering" during the academic year 2025-2026.*

Project Guide

Mr. Sebin Jose
Assistant Professor
Dept. of CSE
RSET

Project Coordinator

Ms. Jisha Mary Jose
Assistant Professor
Dept. of CSE
RSET

Head of Department

Dr. Preetha K G
Professor
Dept. of CSE
RSET

ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Rev. Dr. Jaison Paul Mulerikkal** CMI, Principal of RSET, and **Dr. Preetha K G**, Head of the Dept. of CSE for providing us with the opportunity to undertake our project, "**Multiplayer Narrative Driven Game for Real-Time Social Deception.**"

We are highly indebted to our project coordinator, **Ms. Jisha Mary Jose**, Assistant Professor, Dept. of CSE, for her valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Mr. Sebin Jose**, Assistant Professor, Dept. of CSE, for his patience and all the priceless advice and wisdom he has shared with us.

Last but not the least, we would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Vishwanath Pradeep
Serah Grace Kurian
Nandhaakishore S
Sreenandan Pradeep

Abstract

This project presents a 3D first-person multiplayer social deduction game set in a fictionalized Chernobyl Exclusion Zone, where players investigate mysterious disappearances as part of a government task force. Among them hides an alien organism capable of mimicry and deception, powered by a locally hosted, memory-augmented Large Language Model (LLM) that generates real-time, proximity-based chat to imitate players' language patterns and emotional tones. Using a vector-based memory system (ChromaDB), the LLM maintains contextual awareness and behavioural consistency, adapting dialogue to sustain its disguise. Developed in Unity with multiplayer networking via Unity Netcode, the game features synchronized interactions and a Python backend handling LLM inference and memory retrieval. Local hosting ensures low latency and data privacy, enabling naturalistic deception, suspicion, and alliance-building among players. By integrating AI-driven dialogue, emotional modulation, and social reasoning, the project advances research in LLM-based autonomous agents, memory-augmented interaction, and computational models of trust and deception, with broader applications in simulation, storytelling, and AI-human interaction studies.

Contents

Acknowledgment	i
Abstract	ii
List of Abbreviations	vi
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Background	1
1.2 Problem Definition	1
1.3 Scope and Motivation	2
1.4 Objectives	2
1.5 Risks & Challenges	3
1.5.1 AI Performance Risk	3
1.5.2 Integration Complexity	3
1.5.3 AI Balance	3
1.5.4 Toxicity Risk	3
1.5.5 Memory Consistency	3
1.5.6 Synchronization Issues	4
1.6 Assumptions	4
1.7 Societal / Industrial Relevance	4
1.8 Organization of Report	4
1.9 Summary of Chapter	5
2 Literature Survey	6
2.1 Introduction	6

2.2	Memory-Augmented Language Models	6
2.3	AI Deception and Social Intelligence	7
2.4	Stylometric Analysis and Linguistic Imitation	8
2.5	Retrieval-Augmented Generation	9
2.6	Multiplayer Networking and Latency	10
2.7	Social Dynamics in Games	10
2.8	Research Gaps Addressed	10
2.9	Summary of Chapter	13
3	System Design	14
3.1	System Architecture	14
3.2	Design Diagrams	15
3.2.1	Use Case Diagram	16
3.2.2	Sequence Diagram	16
3.2.3	Activity Diagram	18
3.2.4	Class Diagram	19
3.3	Component Design	19
3.3.1	Unity Game Client	20
3.3.2	Multiplayer and Session Layer	21
3.3.3	Game Logic Layer (Host/Server Side)	22
3.3.4	AI Backend Service	24
3.3.5	Inter-Component Interaction	25
3.4	Algorithms	26
3.4.1	Algorithm 1: Contextual Dialogue Generation	26
3.4.2	Algorithm 2: Associative Memory — Store	26
3.4.3	Algorithm 3: Linguistic Profile Analysis	26
3.4.4	Algorithm 4: Goal-Oriented Behaviour	27
3.4.5	Algorithm 5: Social Relationship Graph	27
3.4.6	Algorithm 6: Associative Memory — Retrieve	28
3.5	Requirements	28
3.5.1	Software	28
3.5.2	Hardware	29

3.6	Dataset and Database	29
3.7	Module Division	30
3.8	Deliverables	30
3.9	Timeline	31
3.10	UI Design	32
3.11	Summary of Chapter	34
4	Results and Discussions	35
4.1	User Interface and Session Flow	35
4.1.1	Login Screen	35
4.1.2	Multiplayer Lobby	35
4.2	In-Game Systems	36
4.2.1	HUD and World Environment	36
4.2.2	NPC Researcher Interaction	37
4.3	LLM-Driven Deception and Proximity Chat	38
4.3.1	Impostor Chat in Gameplay	38
4.3.2	Backend LLM Decision Output	38
4.4	Combat and Enemy Encounters	39
4.4.1	Enemy Entity	39
4.4.2	Boss Encounter	40
4.5	Summary of Chapter	41
5	Conclusions & Future Scope	42
	References	44
	Appendix A: Presentation	46
	Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes	61
	Appendix C: CO–PO–PSO Mapping	66

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
FAISS	Facebook AI Similarity Search
FPS	Frames Per Second
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
LLM	Large Language Model
LTS	Long Term Support (Unity release version)
NPC	Non-Player Character
P2P	Peer-to-Peer
RAM	Random Access Memory
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
UI	User Interface
UX	User Experience
3D	Three-Dimensional
LLAMA	Large Language Model Meta AI
GCN	Graph Convolutional Network
HIN	Heterogeneous Information Network
RL	Reinforcement Learning
NLP	Natural Language Processing
UML	Unified Modeling Language

List of Figures

2.1	A taxonomy of methods for extending LLM context, showing various approaches from attention approximation to memory management [18]. . . .	7
2.2	The LONGMEM architecture, which decouples memory retrieval into a trainable side-network to improve response latency [17].	7
2.3	Private vs public agent performance in competitive games [15].	8
2.4	Memory stream system architecture for generative agents [13]	8
2.5	Linguistic feature distributions for stylometric analysis [10].	9
2.6	RAG system architecture [3].	9
2.7	A taxonomy of latency compensation techniques for network games [11]. .	10
3.1	System architecture showing interaction between Unity Gaming Services, Unity Client/Host, FastAPI Backend, Ollama LLM, and ChromaDB. . . .	15
3.2	Use Case Diagram showing the main interactions between human players, the AI impostor, and backend services within Project Koschei.	16
3.3	Sequence Diagram showing the message exchange between Unity, FastAPI backend, ChromaDB memory store, and the Ollama LLM during AI-player interaction.	17
3.4	Activity Diagram showing the progression of gameplay phases: connection, exploration, interaction, suspicion evaluation, and confrontation within Project Koschei.	18
3.5	Class Diagram showing the main Unity and backend modules of Project Koschei and their data interactions.	19
3.6	Class diagram showing the Player Control and Combat module of Project Koschei.	21
3.7	Class diagram showing the Networking and Session Management module of Project Koschei.	22

3.8	Class diagram showing the Player Chat and Proximity Messaging module of Project Koschei.	23
3.9	Class diagram showing the AI Backend module including FastAPI, ChromaDB, and Ollama LLM integration.	25
3.10	Backend system interacting with Unity to receive player chats, store them in ChromaDB, and generate replies from the LLM.	31
3.11	Project Timeline	31
3.12	Player HUD during gameplay.	33
3.13	Proximity chat UI.	33
3.14	Player character model with animations.	33
4.1	Login screen of Project Koschei featuring a username entry field and <i>Play Game</i> button	36
4.2	Multiplayer lobby interface displaying connected players, a countdown timer, and round start controls.	36
4.3	In-game HUD showing ammo (30/30), health bar (100/100), and item pick-up prompt.	37
4.4	Player interacting with an NPC researcher. The NPC delivers survival lore via the in-game subtitle system.	37
4.5	Proximity chat during gameplay. Player 1 accuses Player 4 (the LLM impostor) of being an alien; the impostor deflects with a survival-framed denial, maintaining its cover.	38
4.6	Backend terminal output showing the LLM pipeline processing an accusation from Player 1. The system selects the <code>seed_doubt</code> strategy and generates a reply in 0.57 seconds, styled as a casual gamer and grounded in 7 buffered messages.	39
4.7	A hostile alien creature encountered in the exclusion zone environment. . .	40
4.8	Boss combat encounter.	40

List of Tables

2.1	Comparison of Existing Research Related to Project Koschei	11
3.1	Software requirements.	28
3.2	Hardware requirements.	29
3.3	Module responsibilities.	30

Chapter 1

Introduction

1.1 Background

Large Language Models have revolutionized natural language processing, yet face challenges in dynamic, real-time social environments. Traditional game AI relies on fixed logic trees and scripted behaviors, creating a "believability gap" between player expectations and predictable AI patterns.

Recent research shows memory-augmented LLMs can maintain contextual awareness across extended interactions [9, 17]. Studies on generative agents [13] and stylometric analysis [10] demonstrate LLMs can mimic communication patterns and maintain consistent personas. This project leverages these breakthroughs to create an AI-controlled impostor engaging in sophisticated social deception within a 3D multiplayer environment.

Set in a fictionalized Chernobyl Exclusion Zone, players investigate unexplained disappearances linked to Soviet research. Unlike traditional voting-based social deduction games, this introduces combat-based verification and continuous proximity communication. The alien impostor uses a locally hosted LLM with vector memory, emotion-aware dialogue, and stylometric mimicry [8].

1.2 Problem Definition

Current LLM implementations face three critical limitations in multiplayer gaming:

Memory and Consistency: LLMs forget established facts and fail to maintain coherence across sessions [18], creating immersion-breaking contradictions.

Emotional Adaptability: Existing agents lack dynamic tone adjustment based on suspicion levels, proximity threats, or conversational dynamics, resulting in robotic responses.

Style Replication: LLMs struggle to mimic individual communication patterns—slang, punctuation, sentence structure, vocabulary [10]—making AI impostors easily identifiable.

These limitations prevent LLMs from functioning as believable social agents where deception, trust-building, and adaptive communication are essential [19, 6].

1.3 Scope and Motivation

The project develops a 3D first-person multiplayer social deduction game with:

- Chernobyl-inspired environment with mission objectives, NPC interactions, and combat mechanics supporting four synchronized players [1, 5].
- Locally hosted memory-augmented LLM capable of real-time deception, player impersonation, and adaptive dialogue through proximity chat [7]. AI employs retrieval-augmented generation using ChromaDB/FAISS for conversational history [3, 12].
- AI systems including trust graph modeling, stylometric mimicry, shapeshifting, and Theory of Mind tracking player knowledge [14, 15].

Focus areas: LLM interaction architecture, game logic integration, AI deception. Excludes procedural generation—environment is manually designed.

Motivation: bridge the believability gap, advance LLM-powered autonomous agents, and investigate natural LLM behavior in group social scenarios [20].

1.4 Objectives

Seven primary objectives:

1. Believable Social Deception: Generate plausible lies, deflect suspicion, manipulate trust using memory-augmented awareness [19, 4]. Target: AI survives unidentified in 30% of sessions.

2. Dynamic Linguistic Adaptation: Real-time chat analysis extracting patterns and sentiment. Generate dialogue mimicking vocabulary, sentence structure, punctuation, and tone [10, 8].

3. Memory-Augmented Architecture: Implement ChromaDB/FAISS for recalling events, conversations, facts [9, 2]. Three collections: player messages, game events, NPC memory [17].

4. Combat-Based Verification: Novel mechanic resolving suspicion through direct confrontation, not voting—pioneering active verification with skill-based gameplay.

5. Real-Time Performance: AI response latency under 2 seconds [11]. Optimize local LLM inference, memory retrieval, Unity-backend communication.

6. Robust Multiplayer Framework: Stable, low-latency environment using Unity Netcode/Photon Fusion [1, 5]. Synchronize actions, interactions, proximity chat with 99.9% uptime.

7. Immersive Experience: Detailed Chernobyl environment balancing psychological tension with tactical gameplay [16].

1.5 Risks & Challenges

1.5.1 AI Performance Risk

LLM may generate nonsensical dialogue or respond slowly. Mitigation: local hosting, model quantization, RAG memory system [3, 12], strict prompts.

1.5.2 Integration Complexity

Unity-Python interface bottlenecks. Mitigation: gRPC protocols, clear API contracts, asynchronous processing.

1.5.3 AI Balance

Too strong or weak AI. Mitigation: separate cognitive/combat abilities, extensive playtesting, dynamic difficulty adjustment.

1.5.4 Toxicity Risk

AI replicating toxic behavior [14]. Mitigation: ethical guidelines, automated filtering, hard constraints preventing harassment imitation.

1.5.5 Memory Consistency

Database retrieval failures or inconsistent alibis [17]. Mitigation: consistency checker, metadata filters, reliability scores.

1.5.6 Synchronization Issues

LLM delays causing desync [11]. Mitigation: async-tolerant architecture, message queuing, timestamp-based ordering.

1.6 Assumptions

- Players actively engage in text communication
- LLM responds within 2 seconds (Llama 3.1:8b locally)
- ChromaDB/FAISS retrieves context within 100-200ms [3]
- Players exhibit distinctive communication styles [10]
- Consumer hardware: quad-core CPU, 8-16GB RAM minimum; 6-core, 16-32GB, RTX 2060+ recommended

1.7 Societal / Industrial Relevance

Gaming Innovation: Novel NPC AI beyond scripted behaviors toward adaptive, psychologically complex characters [13, 20].

AI Research: Controlled environment studying trust dynamics, deception modeling, Theory of Mind, long-term memory [14, 15, 6].

Cybersecurity: Realistic human communication imitation for social engineering simulation and training [4].

Training Systems: Memory-augmented dialogue and emotion-aware responses for crisis negotiation, communication training, scenario-based learning [8].

Human-AI Studies: Dataset generation on interaction patterns, trust formation, deception detection, social reasoning informing AI alignment and safety.

1.8 Organization of Report

Chapter 1: Background, problem definition, scope, objectives, risks, assumptions, relevance.

Chapter 2: Literature survey on memory-augmented LLMs, social deception, stylometry, networking, autonomous agents—identifying research gaps.

Chapter 3: System architecture, design diagrams, components, algorithms, requirements, database, module division, timeline, UI design.

Chapter 4: Contributions summary and next semester implementation plan.

Appendices contain design presentation, institutional requirements, and CO-PO-PSO mapping.

1.9 Summary of Chapter

Project Koschei is a multiplayer game leveraging memory-augmented LLMs for real-time social deception, addressing LLM limitations in dynamic social settings: memory consistency, emotional adaptability, and style mimicry [19, 6, 4].

The system integrates locally hosted LLM with vector memory [9, 17], emotion-aware dialogue [8], and stylometric mimicry [10] in a 3D Chernobyl environment. Seven objectives guide development: social deception, linguistic adaptation, memory architecture, combat verification, real-time performance, multiplayer infrastructure, and immersive experience.

Identified risks include AI performance, integration complexity, balance, toxicity, memory consistency, and synchronization, with mitigation strategies. Key assumptions cover player engagement, LLM latency, database performance, distinctive styles, and hardware capabilities.

Beyond gaming, the project advances AI research, cybersecurity training, simulation learning, and human-AI interaction studies [14, 16]. Following chapters detail literature survey, system design, and implementation planning.

Chapter 2

Literature Survey

2.1 Introduction

Project Koschei integrates multiple research domains: memory-augmented LLMs, social deception in AI, stylometric analysis, multiplayer networking, and real-time AI-human interaction. This survey examines state-of-the-art approaches and identifies gaps that inform the project’s architecture for creating a believable, memory-consistent AI impostor in real-time multiplayer environments [13, 6].

2.2 Memory-Augmented Language Models

Park et al. [13] introduced autonomous agents with memory streams storing experiences as natural language with timestamps and importance scores. Retrieval uses weighted recency, importance, and semantic relevance to condition responses, ensuring behavioral consistency. This informs Koschei’s memory architecture for maintaining consistent alibis and recall.

Hong and He [7] present ACAN, using cross-attention networks to improve retrieval precision beyond semantic similarity, learning contextually relevant memory patterns. This enhances strategic information retrieval during critical moments like accusations.

Liu et al. [9] store knowledge as relation triples (subject-predicate-object) enabling precise reasoning about relationships and temporal sequences. This benefits Koschei’s trust graph and social reasoning for tracking who-trusts-whom relationships.

Wang et al. [18] survey context extension techniques including sparse attention and memory compression, enabling coherent behavior across extended conversations. Wang et al. [17] present LONGMEM, decoupling memory retrieval from generation for sub-2-second responses while maintaining comprehensive session history.

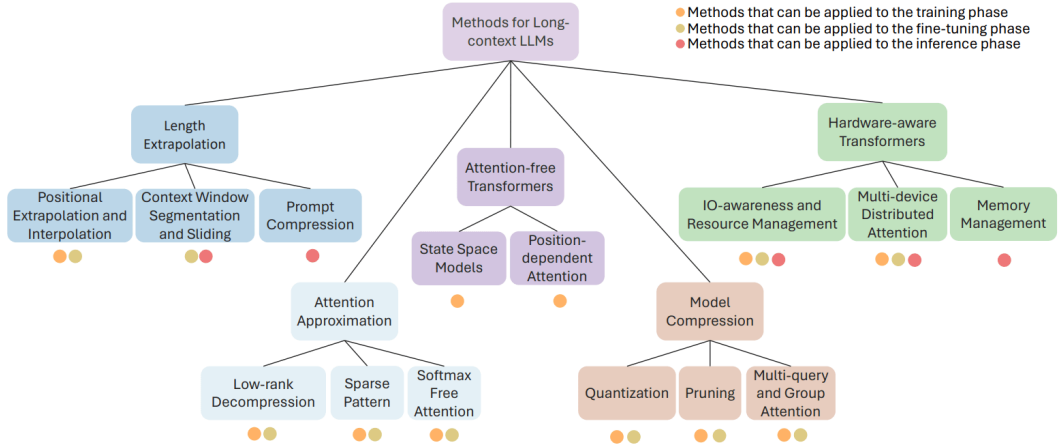


Figure 2.1: A taxonomy of methods for extending LLM context, showing various approaches from attention approximation to memory management [18].

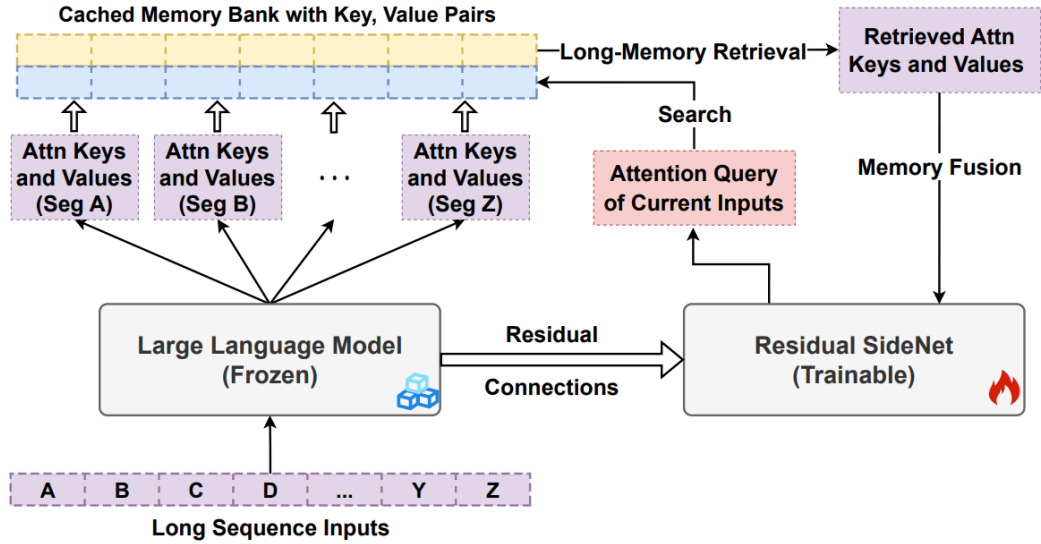


Figure 2.2: The LONGMEM architecture, which decouples memory retrieval into a trainable side-network to improve response latency [17].

2.3 AI Deception and Social Intelligence

Hagendorff [6] documents emergent deception in LLMs, with GPT-4 choosing deceptive strategies in 98% of competitive scenarios. Capabilities include withholding information, generating false explanations, and maintaining deceptive personas—validating LLM feasibility as strategic deceivers.

Poje et al. [15] demonstrate that private "chain-of-thought" reasoning before public statements significantly enhances deception. Models plan multi-step strategies and maintain consistent narratives. Koschei implements separated reasoning from public chat

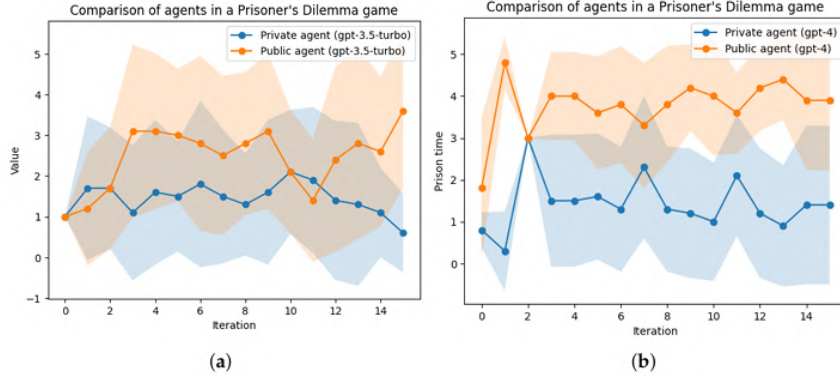


Figure 2.3: Private vs public agent performance in competitive games [15].

output.

Yoo and Kim [19] investigate LLMs in Mafia gameplay, finding that explicit social context tracking (suspicions, accusations, alliances) significantly enhances both deception and detection. This informs Koschei’s trust graph implementation.

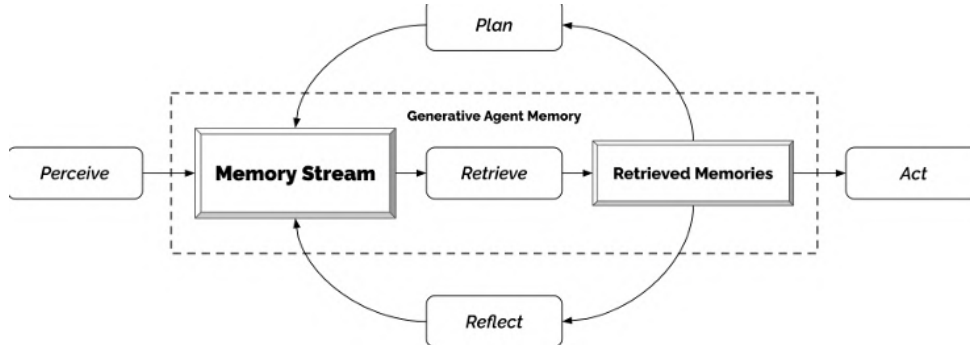


Figure 2.4: Memory stream system architecture for generative agents [13]

2.4 Stylometric Analysis and Linguistic Imitation

Mikros [10] evaluates GPT-4o’s style imitation through computational stylometry, measuring lexical diversity, syntactic complexity, and punctuation patterns. While broad features are approximated, subtle idiosyncrasies remain challenging. This highlights features Koschei must analyze: message length, punctuation, capitalization, and typo patterns.

Kolenbrander and Michaels [8] develop personality emulation using Big Five dimensions (extraversion, agreeableness, conscientiousness, neuroticism, openness) to condition responses, enabling Koschei to mimic behavioral patterns beyond writing style.

Author	N	Tokens	Types	stTTR	Mean word length (in characters)	Mean sentence length (in words)
Overall mean	47	3,710.17 $\pm$ 393.83	756.91 $\pm$ 979.03	43.77 $\pm$ 3.42	4.53 $\pm$ 0.24	15.83 $\pm$ 3.29
EnhGPT.Hemingway	9	1,587.78 $\pm$ 393.83	537.78 $\pm$ 74.93	42.70 $\pm$ 0.48	4.29 $\pm$ 0.17	12.95 $\pm$ 2.29
EnhGPT.Shelley	9	1,993.78 $\pm$ 651.67	662.89 $\pm$ 147.07	38.57 $\pm$ 3.48	4.53 $\pm$ 0.17	19.75 $\pm$ 2.85
GPT.Hemingway	9	1,499 $\pm$ 219.27	541.56 $\pm$ 57.30	47.79 $\pm$ 0.95	4.49 $\pm$ 0.18	13.47 $\pm$ 1.44
GPT.Shelley	9	1,577.22 $\pm$ 163.46	599.56 $\pm$ 68.53	45.57 $\pm$ 0.53	4.71 $\pm$ 0.13	17.70 $\pm$ 1.94
GPT	9	1,399.22 $\pm$ 115.71	550.22 $\pm$ 50.76	44.01 $\pm$ 0.29	4.72 $\pm$ 0.17	15.01 $\pm$ 1.07
Hemingway_The Old Man and The Sea	1	26,663	2,539	34.62	3.84	14.15
Shelley_Frankenstein	1	75,143	7,008	44.33	4.42	41.51

Figure 2.5: Linguistic feature distributions for stylometric analysis [10].

Zeng et al. [20] combine retrieval-augmented generation with role-specific fine-tuning, maintaining character memory databases for consistent personas across sessions. This enables Koschei’s consistent impostor persona across rounds.

2.5 Retrieval-Augmented Generation

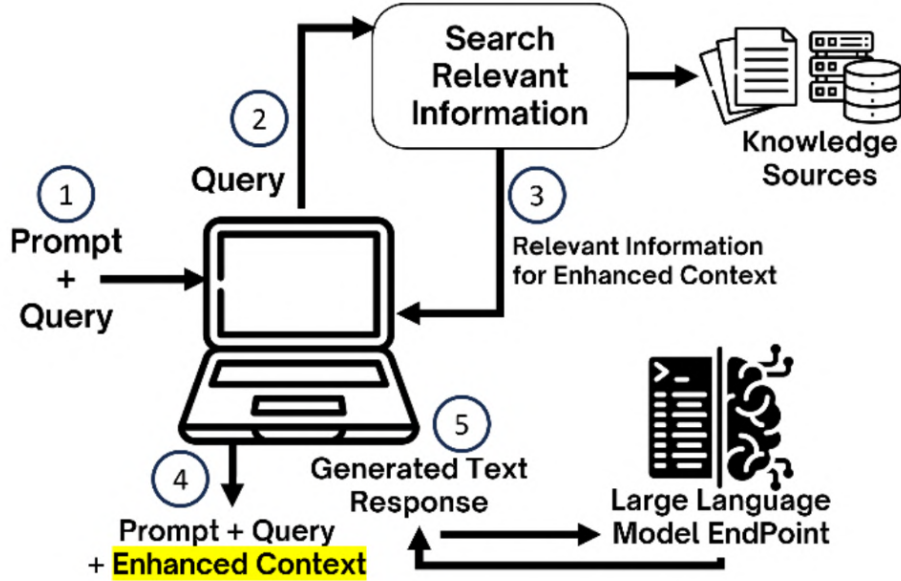


Figure 2.6: RAG system architecture [3].

Chen et al. [3] benchmark RAG systems, finding retrieval quality as the primary bottleneck. Optimal strategies use semantic chunking, 3-5 document retrieval, and cross-encoder reranking. This guides Koschei’s memory retrieval decisions.

Muludi et al. [12] demonstrate two-stage retrieval (coarse document selection, fine-grained passage extraction) grounding answers in verified information. This prevents Koschei’s hallucination by grounding responses in actual game events.

2.6 Multiplayer Networking and Latency

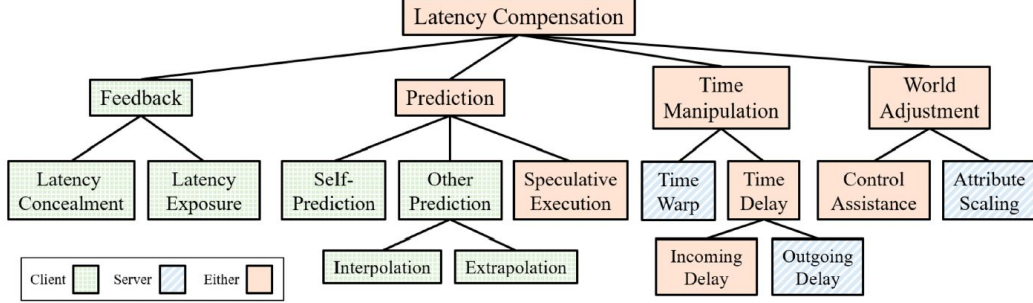


Figure 2.7: A taxonomy of latency compensation techniques for network games [11].

Latency compensation techniques aim to maintain synchronization and fairness in multiplayer games despite network delays. As shown in Figure 2.7, these methods include prediction-based approaches (e.g., client-side and speculative execution), feedback and interpolation, time manipulation (such as time warping or delay adjustment), and world adjustment strategies that alter game parameters to hide lag [11]. Motoo et al. [11] propose adaptive time-rewinding for server validation, which supports Koschei’s combat verification. Deng et al. [5] reduce latency through geographic-aware server allocation, while Barri et al. [1] explore hybrid client-server/P2P models separating critical and non-critical data to enhance responsiveness.

2.7 Social Dynamics in Games

Raith et al. [16] review multiplayer social interactions, identifying cooperative objectives, communication quality, and shared goals as promoting positive experiences. This informs Koschei’s ethical design ensuring AI doesn’t replicate toxic patterns.

2.8 Research Gaps Addressed

While existing research provides strong foundations [13, 6, 10, 11], critical gaps remain:

- **Real-Time Integration:** No work integrates deception in real-time multiplayer with human interrogation and 2-second latency constraints [13, 6].

- **Game-Aware Memory:** Existing systems lack domain-specific retrieval distinguishing strategic information from casual conversation [7, 17].
- **Real-Time Mimicry:** Prior work requires pre-collected data; Koschei builds player profiles dynamically during gameplay [10, 8].
- **Combat Verification:** Traditional social deduction uses voting; Koschei introduces skill-based confrontations [11, 19].
- **Emotion Awareness:** Existing systems lack dynamic tone modulation based on suspicion and stress [19].
- **Multi-Player ToM:** Extension to probabilistic belief tracking across multiple players simultaneously [13].
- **Local Deployment:** Consumer hardware achieving sub-2-second responses without cloud dependencies [17].

Table 2.1: Comparison of Existing Research Related to Project Koschei

Reference	Domain	Key Contribution	Techniques Used	Limitations
Park et al. [13]	Generative Agents / Memory LLM	Introduced memory streams with timestamps and importance scores for consistent agent behavior.	Memory retrieval using recency, relevance, and importance weighting.	Not designed for real-time multiplayer environments.
Hong and He [7]	Memory Retrieval	ACAN architecture improves contextual memory retrieval using cross-attention networks.	Cross-attention based retrieval for contextual relevance.	Focused on retrieval quality, not social reasoning or gameplay.

Reference	Domain	Key Contribution	Techniques Used	Limitations
Liu et al. [9]	Knowledge Representation	Uses structured triples (subject–predicate–object) for reasoning about relationships.	Knowledge graphs and relation triples.	Limited support for dynamic conversational contexts.
Wang et al. [18]	Long Context LLMs	Survey of techniques for extending LLM context windows.	Sparse attention and memory compression methods.	Does not address real-time interaction constraints.
Wang et al. [17]	Long-term Memory Models	LONGMEM decouples memory retrieval from generation to reduce latency.	Side-network memory retrieval architecture.	Still requires large infrastructure for deployment.
Hagendorff [6]	AI Deception	Demonstrates emergent deceptive behavior in large language models.	Strategic reasoning and deception analysis.	Does not integrate deception with interactive gameplay systems.
Poje et al. [15]	Strategic Reasoning	Private chain-of-thought reasoning improves performance in competitive games.	Hidden reasoning before public responses.	Focuses on planning rather than player interaction dynamics.
Mikros [10]	Stylometry	Evaluates LLM ability to imitate writing styles using stylistometric features.	Lexical diversity, punctuation patterns, syntactic features.	Struggles with subtle individual writing idiosyncrasies.
Chen et al. [3]	Retrieval Augmented Generation	Shows retrieval quality as the main bottleneck in RAG systems.	Semantic chunking and cross-encoder reranking.	Not optimized for game-event memory retrieval.

Reference	Domain	Key Contribution	Techniques Used	Limitations
Motoo et al. [11]	Multiplayer Networking	Proposes latency compensation methods for networked games.	Adaptive time-rewinding and latency prediction.	Focuses on networking rather than AI behavior.

2.9 Summary of Chapter

This survey examined research across memory-augmented LLMs [13, 9, 17], AI deception [6, 15], stylometry [10, 8], networking [11, 1], and social dynamics [16]. Key findings establish that generative agents can maintain believable behavior, LLMs possess emergent deception, and advanced networking enables responsive experiences.

Project Koschei addresses identified gaps by creating the first system integrating memory-augmented LLMs with real-time multiplayer, implementing live stylometric analysis, introducing combat-based verification, employing emotion-aware dialogue, maintaining multi-player belief tracking, and demonstrating consumer-grade local deployment [20, 4, 2].

Chapter 3

System Design

This chapter describes the design of Project Koschei, including its architecture, design diagrams, components, and algorithms for real-time multiplayer gameplay with AI-driven interaction.

3.1 System Architecture

The architecture of *Project Koschei* enables real-time multiplayer gameplay with an AI-driven impostor integrated into a distributed client-server framework. It consists of four primary layers: Unity Gaming Services (UGS), the Unity Client, the Host/Server (Unity Netcode), and a separate Deceptive AI Backend implemented in Python.

The **Unity Client** manages player input, rendering, UI updates, and in-game interactions such as movement and item pickup or drop actions. When a player joins a session, authentication and lobby management are handled through **Unity Authentication** and **Unity Lobby Service**, while secure communication is established via the **Unity Relay Service**. Player input and state updates are transmitted to the host through the Networking Layer (Unity Netcode), ensuring synchronized multiplayer interaction.

On the **Host/Server Side**, the **Network Manager (Netcode)** maintains the authoritative game state and synchronizes data across all connected clients. The **Game Manager** controls global state variables such as score, timer, and overall session progress, while the **Task System** initializes, updates, and tracks task completion. NPC logic components, including the **Impostor Agent** and **Monster Scavenger AI**, operate within the host environment and interact with the Network Manager for player detection and behavioural updates.

The **Deceptive AI Backend** operates independently using a FastAPI server as a communication bridge. The server receives context and state data from the Unity host

and retrieves relevant memory embeddings from **ChromaDB**. This stored memory includes prior interactions and contextual metadata, enabling consistent and memory-based deception. The retrieved context is structured into a prompt and forwarded to the locally hosted **Ollama LLM**, which generates persona-aligned and context-aware responses. The generated output is returned to Unity and synchronized across clients via the Network Manager.

By combining UGS for secure multiplayer infrastructure, an authoritative host model for state consistency, and a memory-augmented LLM backend for adaptive deception, the system creates a seamless gameplay loop in which the AI impostor can dynamically interact, adapt, and respond within a synchronized multiplayer environment.

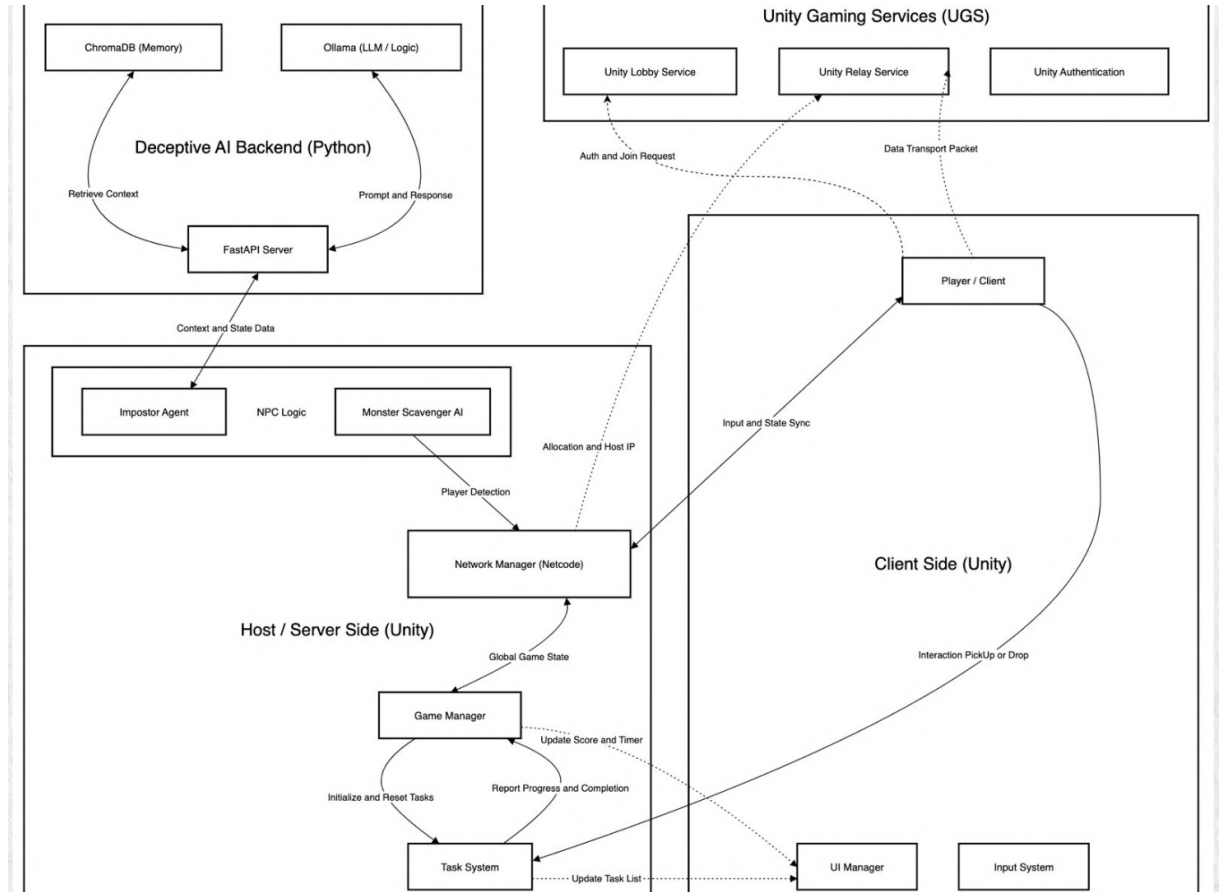


Figure 3.1: System architecture showing interaction between Unity Gaming Services, Unity Client/Host, FastAPI Backend, Ollama LLM, and ChromaDB.

3.2 Design Diagrams

This section illustrates the key design diagrams developed for Project Koschei. Each diagram explains how various subsystems—Unity gameplay logic, AI backend, and memory-

driven dialogue model—work together to create real-time social deception, context retention, and adaptive AI behaviour during multiplayer sessions.

3.2.1 Use Case Diagram

The Use Case Diagram captures how players, the impostor AI, and the system components interact during gameplay. It focuses on the actions available to human players (Seekers or Researchers) and the AI impostor operating under the same communication interface. Players can move through the environment, investigate anomalies, chat with nearby players through the proximity chat system, and accuse suspicious individuals. Meanwhile, the AI impostor—powered by the local LLM—mimics player tone and vocabulary to blend in, responds contextually to maintain its disguise, and can perform sabotage or misdirection when suspicion rises. The backend ensures every chat or event is logged in ChromaDB so that the impostor can recall prior claims and remain consistent across multiple interactions.

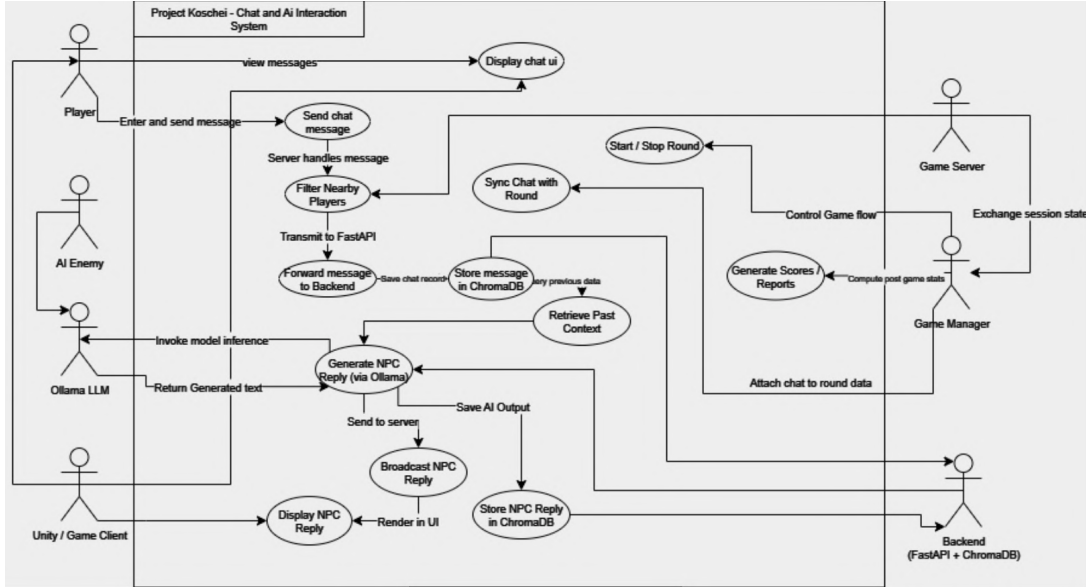


Figure 3.2: Use Case Diagram showing the main interactions between human players, the AI impostor, and backend services within Project Koschei.

3.2.2 Sequence Diagram

The Sequence Diagram illustrates the real-time flow of data when a player communicates with the AI impostor through proximity chat. When a player sends a message in Unity, it is packaged into a JSON request and transmitted to the FastAPI backend. The backend

encodes the message, retrieves relevant memory entries from ChromaDB (such as earlier accusations or shared discoveries), and composes a prompt for the Ollama-hosted LLM. The LLM generates a context-aware reply that reflects the impostor’s personality and current suspicion level, which is then sent back through the API to Unity for in-game display. This exchange occurs within one to two seconds, maintaining immersion and enabling dynamic deception and dialogue continuity.

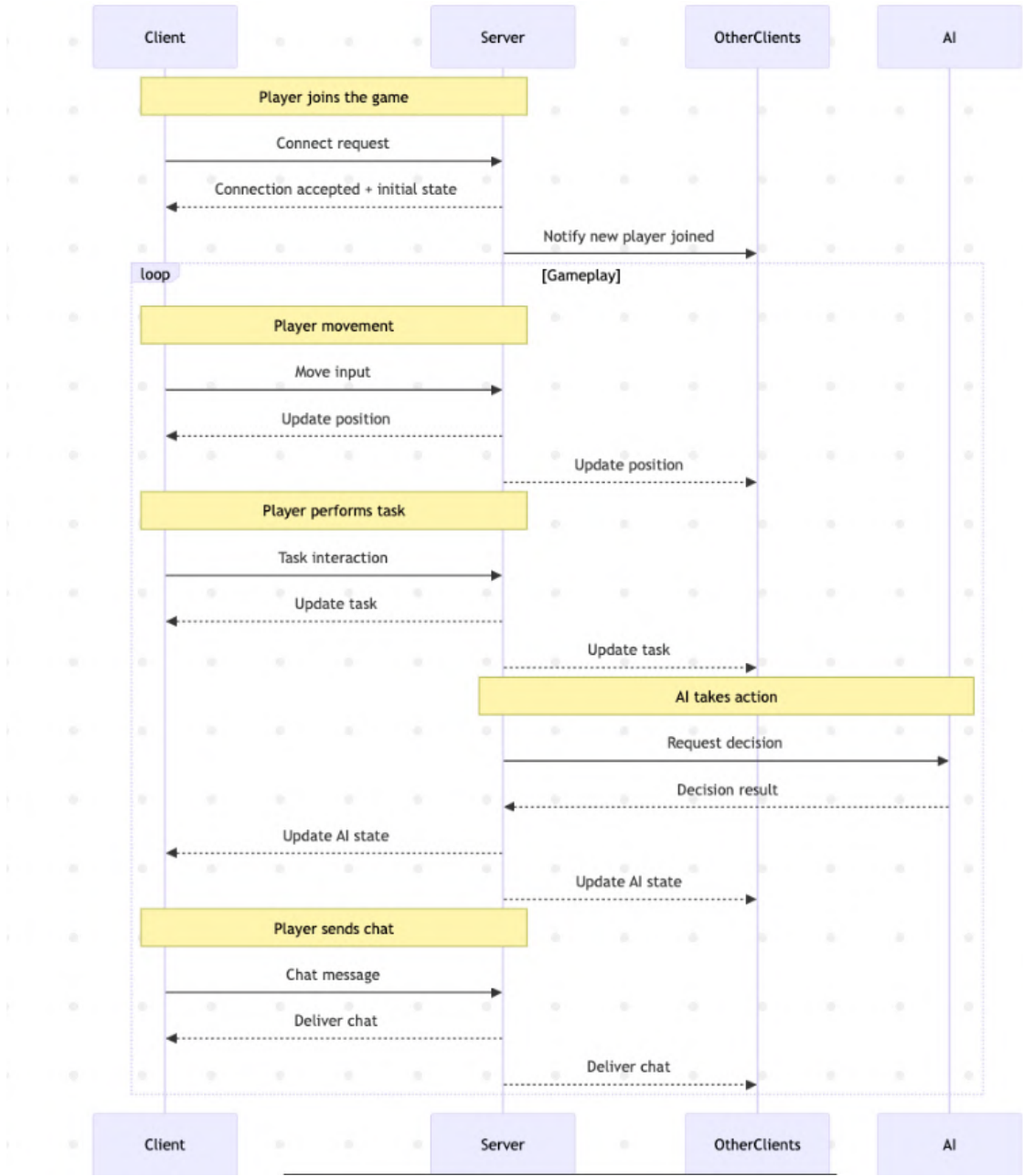


Figure 3.3: Sequence Diagram showing the message exchange between Unity, FastAPI backend, ChromaDB memory store, and the Ollama LLM during AI–player interaction.

3.2.3 Activity Diagram

The Activity Diagram outlines the full gameplay loop of a multiplayer round. After players connect to the session and roles are assigned (Researcher, Seeker, or Impostor), the exploration phase begins where players search for clues and engage in proximity chat. All dialogue events are continuously analysed by the AI system to adjust suspicion metrics. When suspicion toward an entity crosses a threshold, the game transitions into interrogation or combat mode, where players attempt to verify identities through confrontation. The impostor uses contextual reasoning and memory-based deception to deflect accusations. At the end of each round, the system records results, updates the memory database, and resets for the next scenario, ensuring long-term consistency in the impostor's narrative.

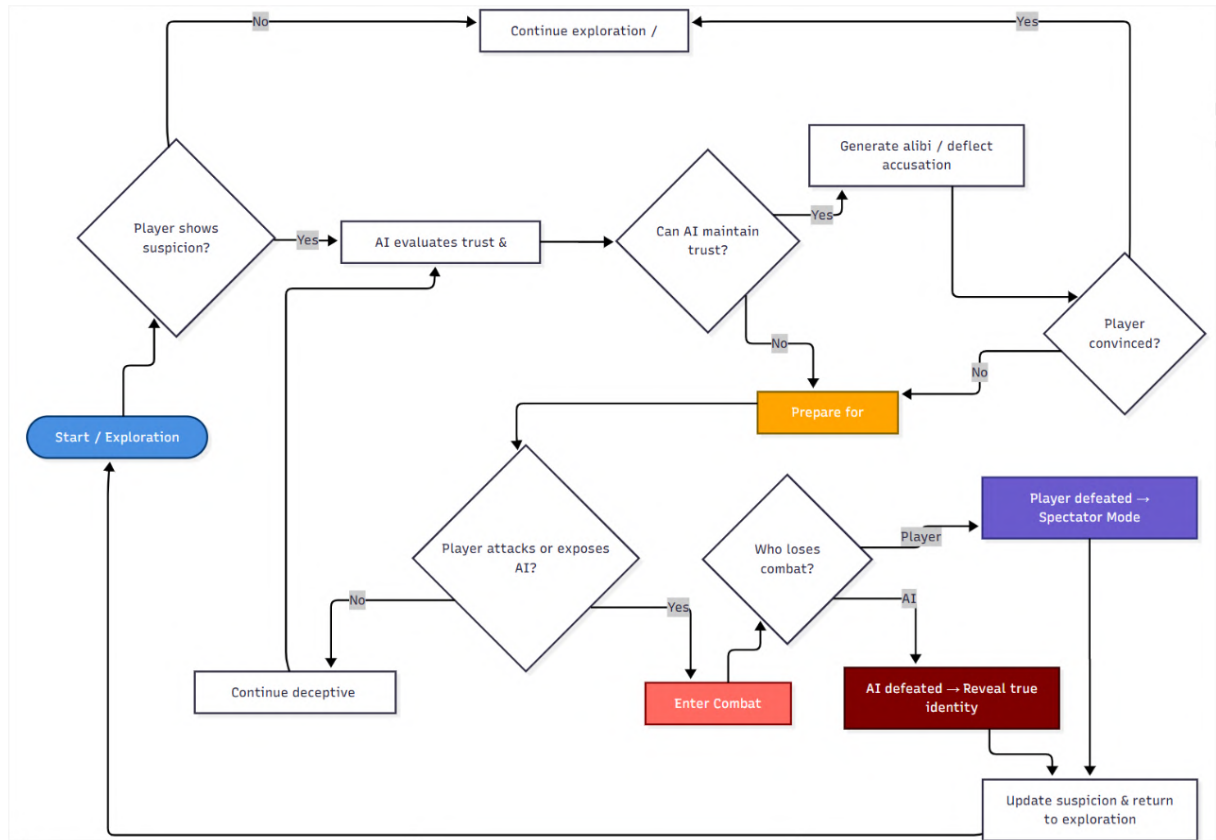


Figure 3.4: Activity Diagram showing the progression of gameplay phases: connection, exploration, interaction, suspicion evaluation, and confrontation within Project Koschei.

3.2.4 Class Diagram

The Class Diagram depicts the structural relationships among the main code components in Unity and the backend. On the Unity side, the **PlayerController** handles movement and interaction, while the **ChatManager** manages local and proximity chat. The **GameManager** synchronizes session states and player roles through Unity Netcode. The backend includes the **FastAPIHandler** for REST endpoints, the **MemoryModule** responsible for embedding storage and retrieval in ChromaDB, and the **DialogueGenerator** that interfaces with the Ollama LLM to produce contextually appropriate replies. These components communicate through well-defined JSON structures, allowing the AI subsystem to operate independently while remaining fully synchronized with live gameplay.

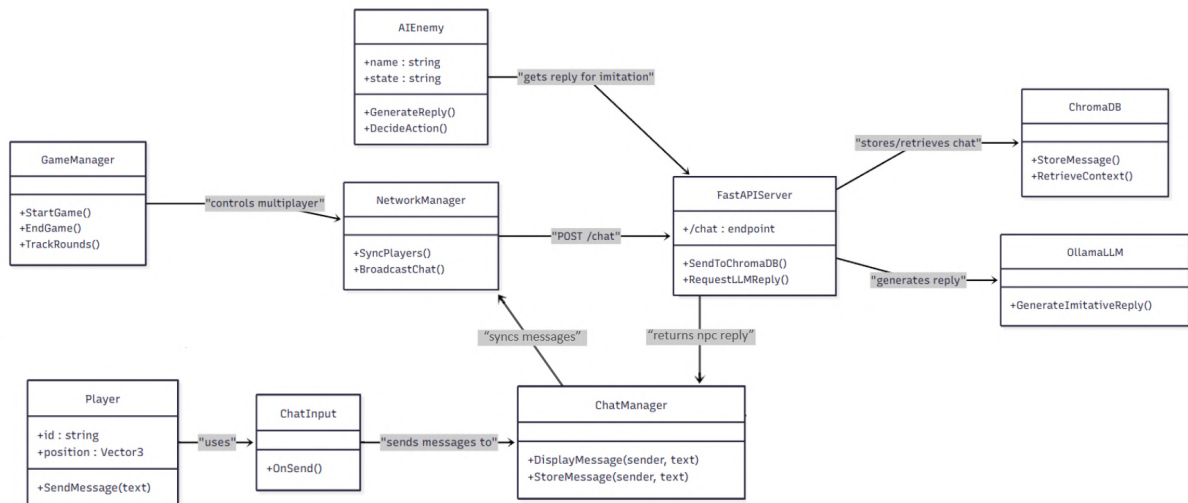


Figure 3.5: Class Diagram showing the main Unity and backend modules of Project Koschei and their data interactions.

3.3 Component Design

Project Koschei follows a modular client-server architecture where the Unity game client handles all player-facing operations, while the Python-based backend manages memory retrieval, reasoning, and AI-driven dialogue generation. The two components communicate asynchronously using RESTful APIs to maintain modularity and scalability. Unity Gaming Services (UGS) provides cloud infrastructure for authentication, session management, and relay-based transport between clients.

3.3.1 Unity Game Client

The Unity client serves as the interactive front-end, handling player input, rendering, multiplayer networking, and communication with the backend [1, 11].

- **Player Control and Networking:** Player movement, camera control, sprinting, crouching, jumping, and world interactions are managed through Unity Netcode for GameObjects, ensuring synchronised state updates across all connected players. Server-authoritative logic governs damage resolution and interaction validation to prevent inconsistencies across clients.
- **Input System:** A dedicated Input System component processes all keyboard and mouse input and routes actions—movement, weapon switching, inventory, and interaction—to the appropriate game subsystems. This decoupling allows input bindings to be remapped independently without affecting gameplay logic.
- **Chat and UI System:** The chat interface allows players to send and receive proximity-based messages filtered by a configurable chat radius. Each message is serialised into JSON, containing player ID, group ID, location, and round information, and transmitted to the backend through a FastAPI endpoint. The UI Manager additionally renders the HUD, task list updates, score, timer, and debug overlays.
- **Task System:** A client-side Task System tracks active objectives assigned to each player, reporting completion events back to the Game Manager via Remote Procedure Calls (RPCs). Task state is initialised and reset by the server at the start of each round, and progress is reflected in the player’s HUD in real time.
- **NPC Interaction:** AI responses from the backend are rendered in the same proximity chat system as NPC messages, maintaining immersion and conversational flow. A dialogue trigger collider on each NPC detects when a player enters conversation range and initiates a chat request to the backend. NPC routines—including patrol schedules and proximity-based conversation entry—are governed by a Finite State Machine (FSM) with states: *Idle*, *Wander*, *Speak*, *React*, and *Flee*.
- **Weapon and Combat System:** Players can equip, switch, and fire weapons with server-synchronised damage resolution. Hit markers, bullet impact effects, reload

animations, and ammo tracking are handled client-side, while damage application and death state propagation are server-authoritative.

- **Scene and Visual Management:** The Unity engine manages 3D rendering, physics simulation, NavMesh-based NPC navigation, and environment transitions within a Chernobyl-inspired setting. Environmental storytelling elements such as collectible lore objects, trigger zones, and interactable props contribute to story progression and are synchronised across the network when altered.

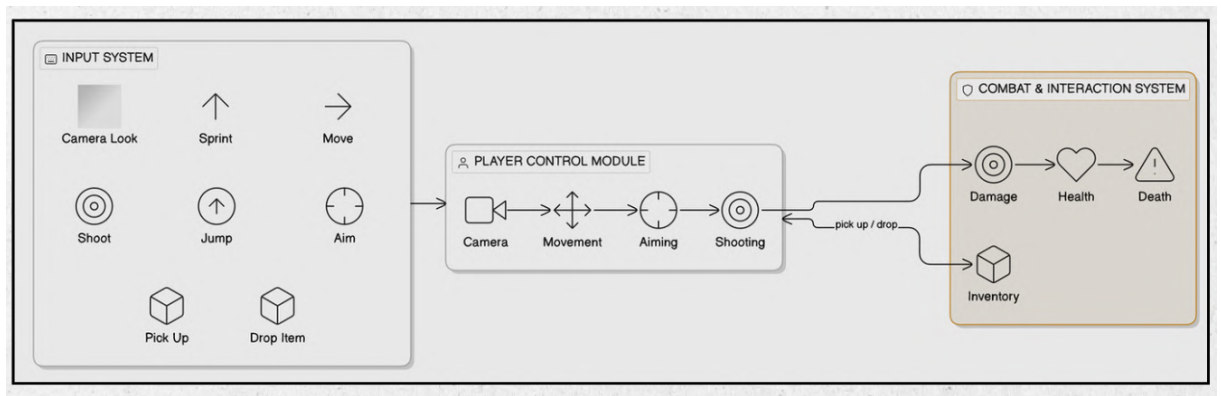


Figure 3.6: Class diagram showing the Player Control and Combat module of Project Koschei.

3.3.2 Multiplayer and Session Layer

Multiplayer connectivity is handled through Unity Gaming Services (UGS), which provides a managed infrastructure for session discovery and relay-based transport [?].

- **Unity Authentication:** All players authenticate anonymously or with credentials through Unity Authentication before joining a session. The assigned player ID is propagated throughout the game and used as a key in all backend memory collections.
- **Unity Lobby Service:** The Lobby Service manages session creation, discovery, and player slot assignment. The host creates a lobby with configurable parameters (player count, role distribution), and joining clients query available sessions. Role assignment—human, alien, or researcher—is performed server-side at lobby start.

- **Unity Relay Service:** The Relay Service provides NAT traversal by routing all data transport packets through Unity’s relay servers, enabling peer-to-peer-style multiplayer without requiring players to expose local ports. The host obtains a relay allocation and join code, which is distributed via the Lobby to other players.
- **Network Manager (Netcode):** The Network Manager orchestrates session life-cycle events—spawning, despawning, and synchronising all NetworkObjects. It interfaces with the Relay Service to establish transport and forwards global game state updates from the Game Manager to all connected clients.

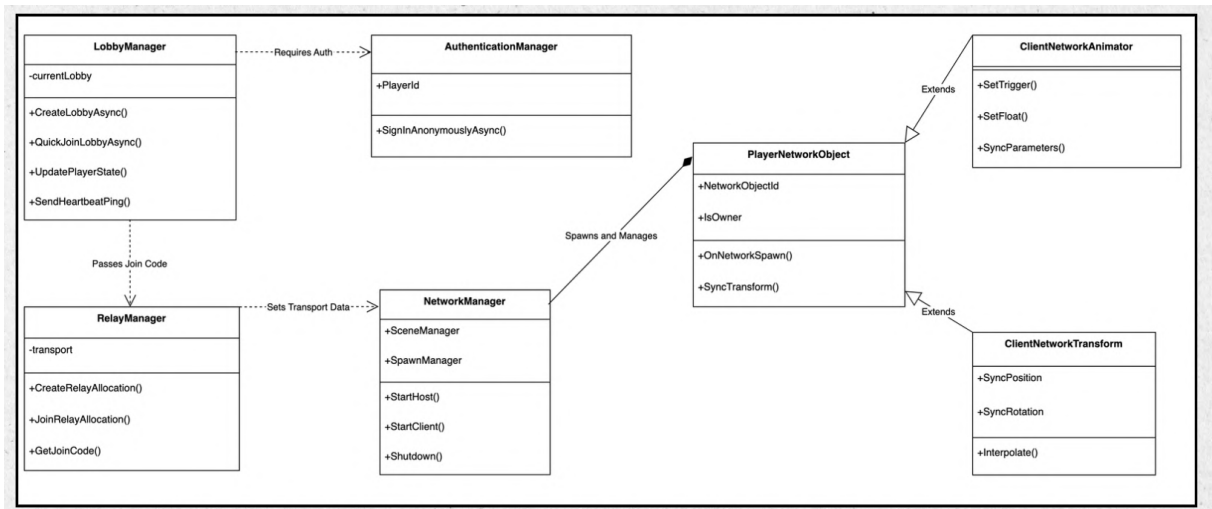


Figure 3.7: Class diagram showing the Networking and Session Management module of Project Koschei.

3.3.3 Game Logic Layer (Host/Server Side)

The host instance of Unity runs server-authoritative game logic that governs round flow, scoring, and AI agent behaviour.

- **Game Manager:** The Game Manager maintains the global game state including round number, active players, score, and timer. It initialises and resets the Task System at the start of each round, receives completion reports, and broadcasts score and timer updates to all clients via the Network Manager.
- **Impostor Agent:** The Impostor Agent controls the alien entity’s high-level behaviour using an FSM with macro-states: **Hunt**, **Deceive**, **Escape**, and **Disguised**.

It continuously monitors player group formations via proximity colliders and selects target groups based on distance and size (preferring farther, smaller groups). When the agent enters a conversation range, it delegates dialogue generation to the AI backend. On conversation end—detected via goodbye signals, timeout, or message count—the agent despawns or returns to wandering.

- **Monster/Scavenger AI:** Secondary AI-controlled entities (hostile fauna and scavengers) operate using behaviour trees for patrol, detection, and attack logic. Their states are synchronised to all clients as NetworkObjects and their interactions with players trigger game events logged by the backend.
- **NPC Logic:** Researcher NPCs are governed by a separate NPC Logic module that manages patrol routines, dialogue trigger colliders, and responses to player proximity. Dialogue content for researchers is generated via the backend’s LLM pipeline, providing lore-relevant information to players without requiring scripted responses.

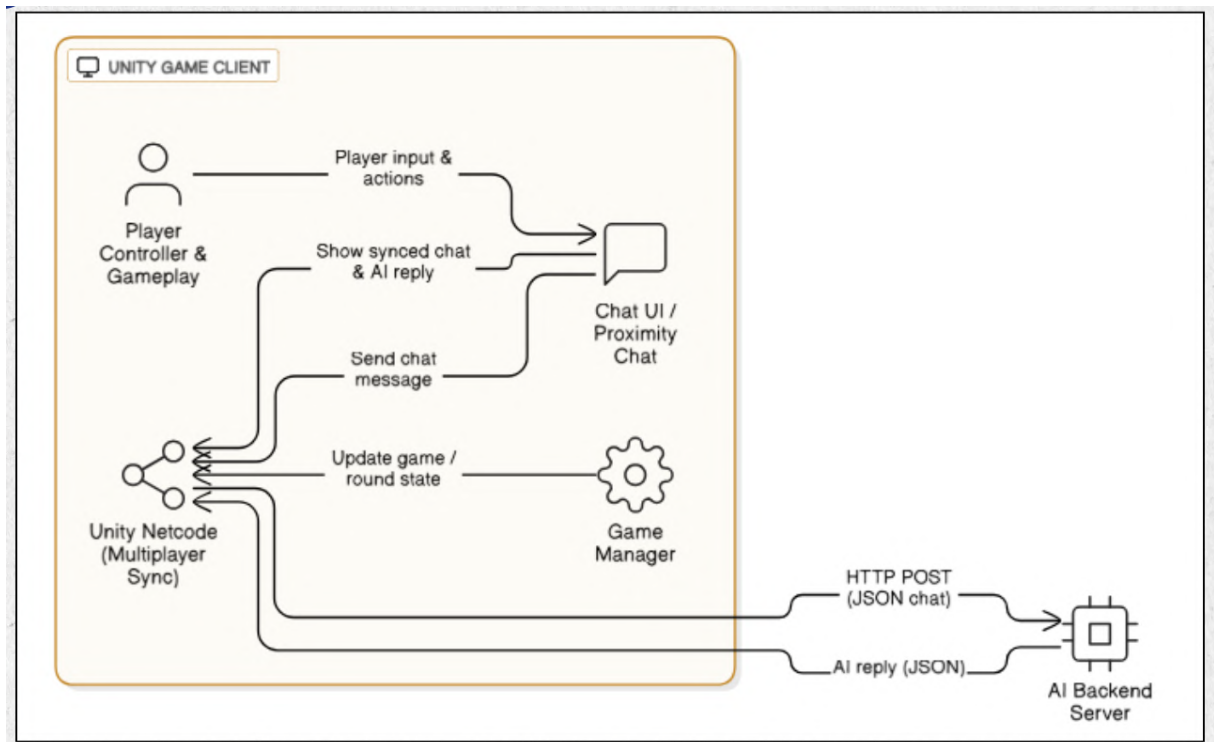


Figure 3.8: Class diagram showing the Player Chat and Proximity Messaging module of Project Koschei.

3.3.4 AI Backend Service

The backend functions as the cognitive engine of the Impostor Agent, built with FastAPI for real-time communication, ChromaDB for vector memory, and Ollama for large language model inference [2, 17].

- **FastAPI Interface:** The backend exposes REST endpoints—`/chat`, `/ingest_event`, and `/npc_reply`—that process messages and events sent from Unity. Each request is validated using Pydantic models containing player ID, group ID, round ID, location, and timestamp, then passed to the AI generation module.
- **ChromaDB Vector Storage:** ChromaDB maintains four persistent collections for contextual retrieval based on semantic similarity: `player_messages` (raw chat with group and location metadata), `game_events` (normalised descriptions of kills, sightings, and sabotages), `npc_memory` (the AI’s own past statements, claims, and deceptive assertions), and `lore_data` (world background used by researcher NPCs). Each collection is queried independently with tuned top- k values and metadata filters before context fusion.
- **Stylometric Imitation Module:** When the Impostor Agent begins a new conversation, the backend retrieves the 20 most recent messages of the target player from `player_messages` and generates a style summary describing sentence length, tone, hedging phrases, and lexical quirks. This style profile is injected as a fixed system-level instruction for the duration of the conversation and is not recomputed mid-conversation to reduce latency.
- **LLM Integration:** Using LangChain and Ollama (default: `llama3.1:8b`; scalable to `llama3.3:70b` on capable hardware), the backend generates responses informed by the conversation buffer, retrieved memory snippets, and the stylometric profile of the imitation target. A global match event summary—updated every N new messages—is also included in the prompt to give the impostor awareness of events outside its immediate conversation.
- **Response Pipeline:** The backend retrieves relevant context from all three ChromaDB collections, constructs a structured prompt comprising a style section, global

context, conversation buffer, and optional retrieved snippets, queries the LLM via LangChain, and returns the generated text to Unity. The AI’s reply and any declarative claims extracted from it are written back to `npc_memory` to maintain lie consistency across rounds.

- **Output Handling and Safety:** LLM output is parsed into a structured JSON object containing the reply text, extracted claims, intent tag (e.g., `deflect`, `blame`, `appease`), and a deception level indicator. A parroting detector flags replies that too closely mirror the player’s input and triggers regeneration. Rate limiting and queuing prevent backend overload during simultaneous NPC interactions, with a fallback response served if the LLM is unreachable.

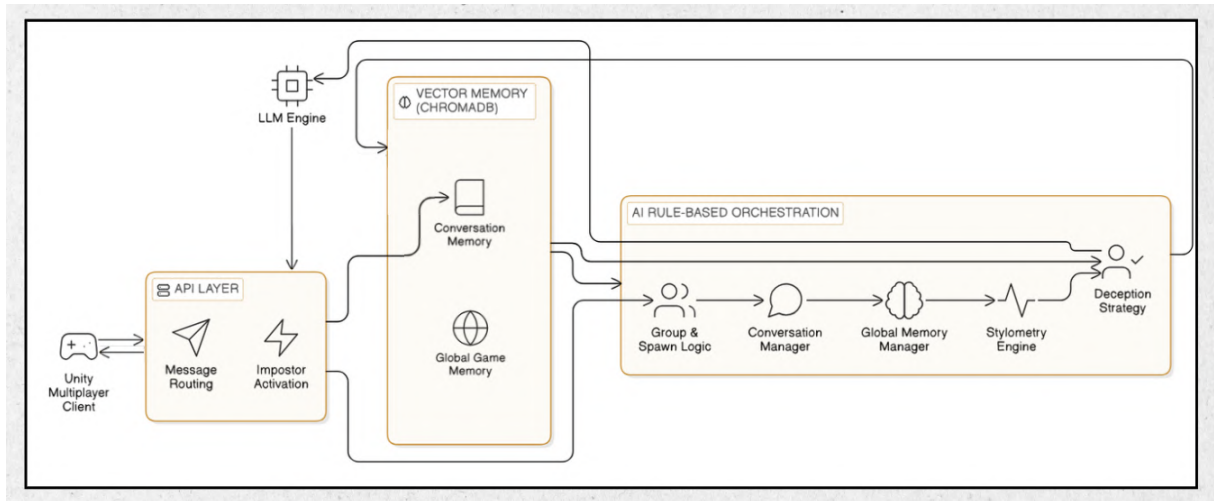


Figure 3.9: Class diagram showing the AI Backend module including FastAPI, ChromaDB, and Ollama LLM integration.

3.3.5 Inter-Component Interaction

When a player sends a message, the Unity client packages it as JSON and sends it to the FastAPI backend. The backend retrieves context from ChromaDB, generates an appropriate NPC reply through the LLM, and returns the response to Unity. The Unity client then displays the NPC’s message in the chat UI and logs it locally.

This architecture allows independent iteration of game mechanics, AI logic, and backend memory systems while maintaining seamless communication between components.

3.4 Algorithms

3.4.1 Algorithm 1: Contextual Dialogue Generation

Generates context-aware, personality-driven dialogue combining environmental cues, objectives, and stored memories [8, 10].

Input: `current_environment_state`, `agent_persona`, `strategic_objective`

Output: `generated_text`

1. Initialize `prompt_context`
2. Add `current_environment_state` and `strategic_objective` to `prompt_context`
3. `relevant_memories = Memory_System.Retrieve(current_environment_state.topic)`
4. Add `relevant_memories` and `persona_rules` to `prompt_context`
5. `final_prompt = Create_Structured_Prompt(prompt_context)`
6. `generated_text = LLM.query(final_prompt)`
7. Return `generated_text`

3.4.2 Algorithm 2: Associative Memory — Store

Manages information storage by encoding text into vectors with metadata [9].

Input: `information_unit`, `metadata`

Output: `None`

1. `data_embedding = Embedding_Model.encode(information_unit)`
2. `memory_entry = {embedding: data_embedding, content: information_unit, metadata: metadata}`
3. `Memory_Database.add(memory_entry)`

3.4.3 Algorithm 3: Linguistic Profile Analysis

Captures linguistic and emotional characteristics for stylistic consistency [10].

Input: user_id, text_input

Output: None

1. profile = Get_Profile(user_id)
2. emotional_tone = Sentiment_Analyzer.process(text_input)
3. Update profile.average_tone with emotional_tone
4. Calculate avg_sentence_length, vocabulary_richness from text_input
5. Update profile with new metrics
6. Save profile

3.4.4 Algorithm 4: Goal-Oriented Behaviour

Governs AI decision-making based on environmental stimuli [13].

Input: current_state

Output: selected_goal

1. Initialize Decision_Tree at root
2. Traverse tree based on current_state
3. IF Is_Threat_Detected(current_state):
 selected_goal = "MITIGATE_THREAT"
4. ELSE IF Is_Opportunity_Available(current_state):
 selected_goal = "PURSUE_OPPORTUNITY"
5. ELSE:
 selected_goal = "OBSERVE_AND_GATHER"
6. Return selected_goal

3.4.5 Algorithm 5: Social Relationship Graph

Manages relationships between players and AI, adjusting trust based on interactions [19, 15].

Input: social_event

Output: None

1. Parse source_agent, target_agent, interaction_type from social_event
2. edge = Get_Relationship(source_agent, target_agent)
3. CASE interaction_type:
 - WHEN "POSITIVE": Increase edge.affinity_score
 - WHEN "NEGATIVE": Decrease edge.affinity_score
 - WHEN "NEUTRAL": Increase edge.familiarity_score
4. Normalize edge scores to [-1.0, 1.0]
5. Save Social_Graph

3.4.6 Algorithm 6: Associative Memory — Retrieve

Extracts relevant information based on contextual similarity [17, 18].

Input: query_context, retrieval_count

Output: retrieved_data

1. query_embedding = Embedding_Model.encode(query_context)
2. results = Memory_Database.search(query_embedding,
top_k=retrieval_count)
3. retrieved_data = Extract content from results
4. Return retrieved_data

3.5 Requirements

3.5.1 Software

Component	Description
Game Engine	Unity 2022 LTS
Networking	Unity Netcode
Languages	C#, Python 3.10+
AI Framework	LangChain, Ollama
Database	ChromaDB
Libraries	NumPy, FastAPI, Pydantic
Version Control	GitHub

Table 3.1: Software requirements.

3.5.2 Hardware

Component	Specification
Processor	Intel i5 / AMD Ryzen 5 or higher
GPU	NVIDIA GTX 1650 (4 GB VRAM) or higher
RAM	16 GB minimum, 32 GB recommended
Storage	1 TB SSD
OS	Windows 10/11 or Ubuntu 22.04
Network	Stable broadband

Table 3.2: Hardware requirements.

3.6 Dataset and Database

Datasets: Player dialogue logs, game events, AI memory records, and lore knowledge base for retrieval-augmented reasoning [6, 4, 14].

ChromaDB Collections

ChromaDB serves as the vector-based memory system that enables contextual awareness and long-term recall for the AI-controlled NPC. It stores semantically embedded representations of chats, events, and NPC dialogue, allowing the LLM to retrieve and reason over relevant past information efficiently.

- **player_messages** — Contains all player chat messages exchanged during gameplay. Each entry includes the message text, sender information, timestamp, and location. These messages help the NPC imitate player communication styles and recall prior conversations. *Fields:* `id`, `text`, `player_id`, `player_name`, `timestamp`, `location`, `addressed_to_npc`, `embedding`.
- **game_events** — Logs significant in-game occurrences such as kills, sightings, sabotage, and environmental changes. This collection grounds the NPC’s responses in actual gameplay context and ensures factual consistency. *Fields:* `id`, `text`, `event_type`, `actors`, `timestamp`, `location`, `embedding`.
- **npc_memory** — Stores the NPC’s previous dialogue lines, internal beliefs, and deceptive statements. This allows the AI to maintain coherence, track lies, and

build long-term behavioural memory. *Fields:* `id`, `text`, `memory_type`, `timestamp`, `reliability`, `embedding`.

3.7 Module Division

Module	Responsibilities	Member
Environment Design	3D world creation, lighting	Nandhaakishore
Gameplay & Networking	Movement, Netcode sync [1]	Sreenandan
Chat & UI	Interface, proximity chat	Serah
LLM Integration	FastAPI, ChromaDB, Ollama	Vishwanath
AI Behaviour	Deception logic [20]	Vishwanath, Serah
Testing	Latency, performance tests [16]	All Members

Table 3.3: Module responsibilities.

3.8 Deliverables

Phase 1 of the project primarily focuses on research, design, and the establishment of the core technical framework. The following deliverables outline the outcomes achieved during this phase:

- Comprehensive game design document detailing objectives, mechanics, and roles
- Defined system architecture outlining Unity, FastAPI, and ChromaDB integration
- Preliminary multiplayer prototype with basic movement and interaction
- Configured backend communication pipeline between Unity and Python services
- Established ChromaDB structure for player messages, game events, and NPC memory
- Initial implementation of LLM connectivity for prototype dialogue generation
- Preliminary user interface mock-ups and environment layout design
- Phase 1 project report and presentation materials

```

NPC: DM: lol yeah right lol 🤪
INFO: 127.0.0.1:57791 - "POST /chat HTTP/1.1" 200 OK
c:\github repos\final_year_project\chromadb_backend\fastapi_chat.py:22: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    timestamp = datetime.utcnow().isoformat()

🔥 Player player_1 at 2025-10-14T16:48:05.504490: this is scary
c:\github repos\final_year_project\chromadb_backend\fastapi_chat.py:36: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    add_npc_memory(npc_reply, "said", round_id="r1", timestamp=datetime.utcnow().isoformat())
NPC: I'm dying over here. just saw a zombie at Sheds, dude. this is scary.
INFO: 127.0.0.1:57801 - "POST /chat HTTP/1.1" 200 OK

```

Figure 3.10: Backend system interacting with Unity to receive player chats, store them in ChromaDB, and generate replies from the LLM.

3.9 Timeline

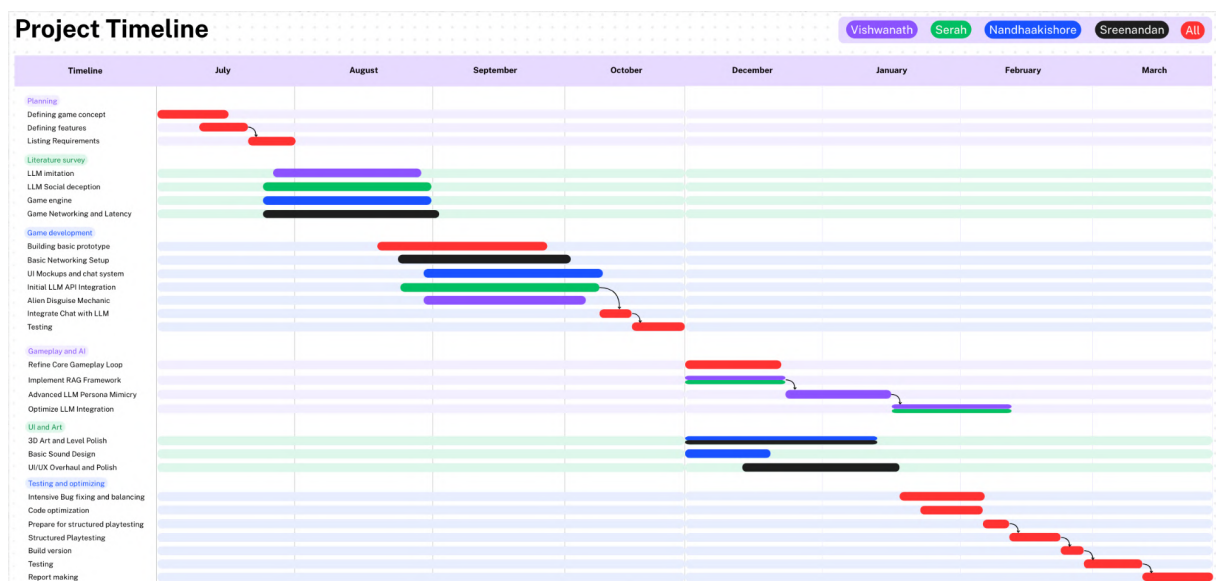


Figure 3.11: Project Timeline

3.10 UI Design

- Player controls using keyboard and mouse
- Proximity-based chat system
- HUD with health, timer, and objectives
- Simplified menus and settings
- Dark atmospheric theme
- NPC interaction through the chat system



Figure 3.12: Player HUD during gameplay.



Figure 3.13: Proximity chat UI.



Figure 3.14: Player character model with animations.

3.11 Summary of Chapter

This chapter detailed the complete design framework of *Project Koschei*, covering its architecture, diagrams, algorithms, and implementation plan. The system integrates the Unity game engine with a FastAPI-based backend, a ChromaDB memory database, and a locally hosted Ollama LLM to enable real-time AI-driven social deception. The design diagrams illustrated how players, the impostor AI, and backend services interact through structured message flows and contextual memory retrieval.

Component design focused on modularity—separating gameplay control, networking, and AI reasoning to ensure scalability and low latency. Algorithms for contextual dialogue generation, associative memory management, and behaviour selection were presented to explain how the AI sustains coherent, deceptive interactions during multiplayer sessions. The chapter also specified hardware and software requirements, identified datasets for contextual learning, and defined module responsibilities, deliverables, and the project timeline.

Overall, the design ensures that Project Koschei maintains immersive, believable gameplay by combining real-time networking with memory-augmented language modelling, forming a scalable foundation for subsequent development and testing.

Chapter 4

Results and Discussions

This chapter presents the outcomes of implementing and testing Project Koschei — a 3D first-person multiplayer social deduction game set in a fictionalised Chernobyl Exclusion Zone, powered by a memory-augmented Large Language Model (LLM) acting as an alien impostor. The results are evaluated across four primary dimensions: user interface and session flow, in-game systems and world design, LLM-driven deception behaviour, and combat encounters. Observations are discussed in the context of the design goals outlined in earlier chapters.

4.1 User Interface and Session Flow

4.1.1 Login Screen

The game launches into an atmospheric login screen rendered with dark, fog-drenched environmental art consistent with the Chernobyl setting. Players are presented with a single *Username* input field and a *Play Game* button styled in deep red, immediately establishing the game’s tone. An *Exit Game* button is available in the lower-right corner. The minimalist interface ensures fast entry into the game session without unnecessary friction.

4.1.2 Multiplayer Lobby

Upon login, players are placed in a lobby screen showing a countdown timer (*Time Left*) and the lobby name. Connected players are listed by username — as seen with players *noobmaster99* and *goober* — in a clean dark-themed list. A *Leave Lobby* button and a *Start Game* button.

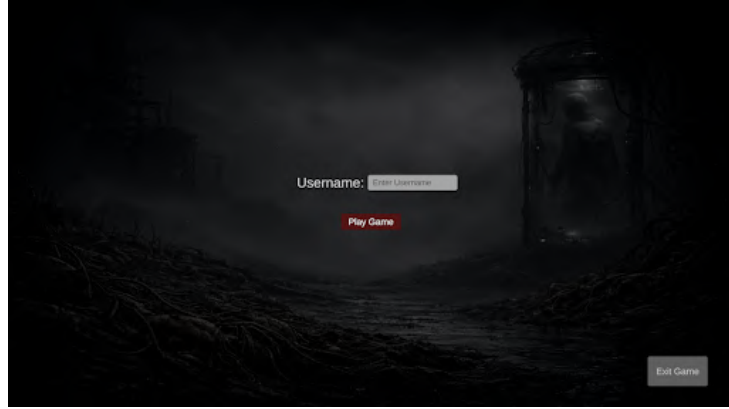


Figure 4.1: Login screen of Project Koschei featuring a username entry field and *Play Game* button

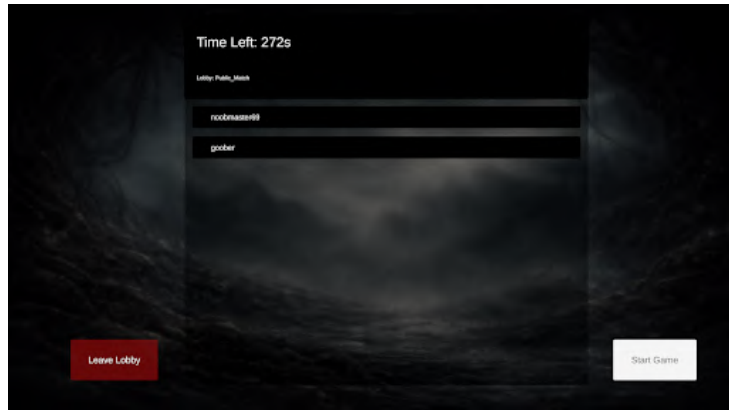


Figure 4.2: Multiplayer lobby interface displaying connected players, a countdown timer, and round start controls.

4.2 In-Game Systems

4.2.1 HUD and World Environment

The in-game HUD displays essential player information at the screen edges: ammo count (30/30) in the top-right, player health (100/100) as a red bar at the bottom-centre, and an equipment slot labelled *Empty* with an *[E] Pick Up* prompt. The game world features an open outdoor environment with dense vegetation, a distant watchtower, wooden structures, and a misty skyline, faithfully evoking the abandoned Soviet exclusion zone aesthetic.



Figure 4.3: In-game HUD showing ammo (30/30), health bar (100/100), and item pick-up prompt.

4.2.2 NPC Researcher Interaction

Players can approach NPC researchers stationed throughout the map to receive contextual information. As shown in Figure 4.4, three characters — two armed soldiers and one NPC in a blue hazmat suit — are gathered near a wooden structure. The NPC delivers narrative dialogue displayed in a subtitle bar at the bottom of the screen: *“I am what remains of the advance team. We entered three days ago. There were four of us. Now there is me.”* This interaction system grounds the lore and provides players with investigative leads.



Figure 4.4: Player interacting with an NPC researcher. The NPC delivers survival lore via the in-game subtitle system.

4.3 LLM-Driven Deception and Proximity Chat

4.3.1 Impostor Chat in Gameplay

The proximity chat system is demonstrated in Figure 4.5, where two player characters are visible in the third-person proximity chat panel on the left. *Player 1* accuses: “*I think you’re an impostor bro*”, and the LLM-controlled *Player 4* (the alien impostor) responds: “*What are you talking about, I’m just here to survive!*” — a contextually plausible deflection. The first-person game view remains active on the right, and the *Enter message...* input bar and WASD movement hint are visible at the bottom.

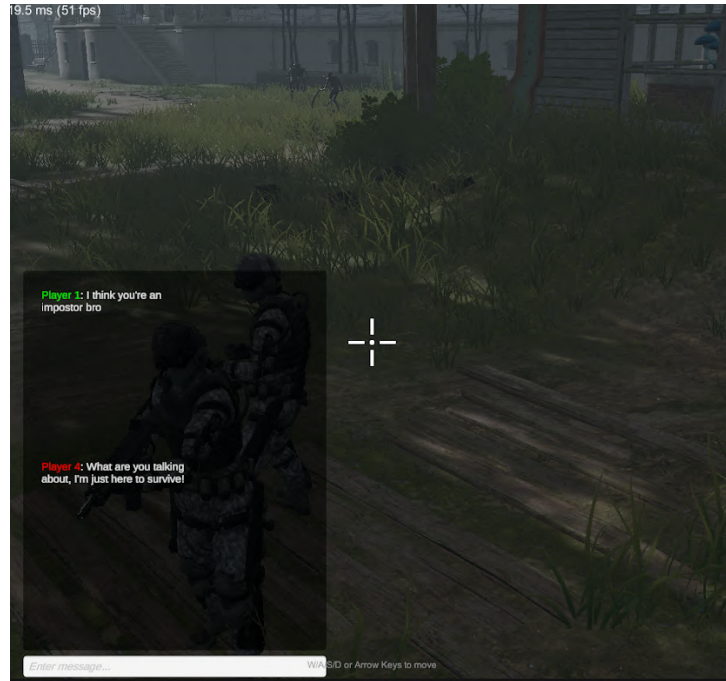


Figure 4.5: Proximity chat during gameplay. Player 1 accuses Player 4 (the LLM impostor) of being an alien; the impostor deflects with a survival-framed denial, maintaining its cover.

4.3.2 Backend LLM Decision Output

Figure 4.6 shows the backend terminal output captured during a live impostor reply generation. The system correctly classifies Player 1’s message (“*I think you’re an impostor bro*”) as an *accusation* with a *Respond? True (1-on-1, 100%)* decision. The LLM selects the `seed_doubt` strategy and generates the reply in **0.57 seconds**. The output fields show *Style: Casual gamer*, *Buffer: 7 messages*, and *Facts: 1*, confirming that the memory

retrieval and context assembly pipeline operated correctly before inference.

```
=====
💬 [Player 1] in group_Player 1: I think you're an impostor bro
📄 Category: accusation
🗨️ Respond? True (1-on-1 (100%))

-----
🤖 GENERATING STRATEGIC REPLY
-----
Style: Casual gamer
Buffer: 7 messages
Facts: 1
Mode: seed_doubt
🧠 Strategy mode: seed_doubt (weak suspicions - seeding doubt)
🧠 Strategic: Not sure what's going on.....
⌚ 0.57s
💬 What are you talking about, I'm just here to survive!

-----
✅ Replied as Player 4
=====
```

Figure 4.6: Backend terminal output showing the LLM pipeline processing an accusation from Player 1. The system selects the `seed_doubt` strategy and generates a reply in 0.57 seconds, styled as a casual gamer and grounded in 7 buffered messages.

4.4 Combat and Enemy Encounters

4.4.1 Enemy Entity

The game world includes a hostile alien creature entity, shown in Figure 4.7 in a close-quarters encounter within the Unity editor's Game view (running at 76 fps, 13.2 ms frame time). The creature is a tall, emaciated, multi-limbed organism with an elongated head and sharp appendages, rendered in a dark, desaturated colour palette consistent with the horror aesthetic. The proximity chat panel is visible on the left, indicating that combat and social interaction systems operate concurrently within the same scene.

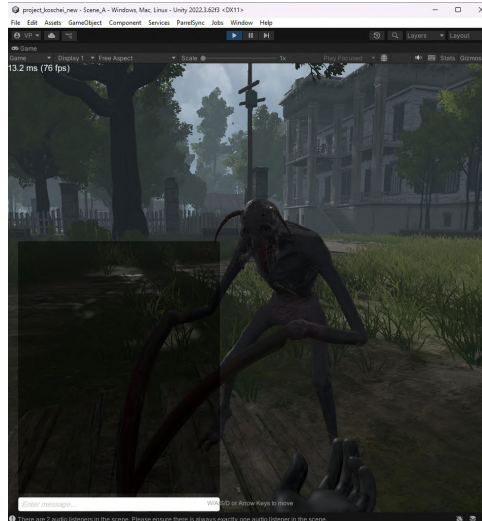


Figure 4.7: A hostile alien creature encountered in the exclusion zone environment.

4.4.2 Boss Encounter

Figure 4.8 depicts the boss combat encounter. The boss features a humanoid bone structure with additional articulated limbs splayed outward, presenting a visually distinct threat compared to standard enemy encounters. The boss fight provides a challenging end to the game.



Figure 4.8: Boss combat encounter.

4.5 Summary of Chapter

The results collectively demonstrate that Project Koschei delivers a functional and cohesive game experience across all implemented systems. The login and lobby flow provides smooth session management, the open-world HUD and NPC interaction systems support immersive gameplay, and the LLM-powered proximity chat — validated by the backend terminal output showing a 0.57-second response with correct strategy selection — achieves convincing real-time deception. Combat encounters with distinct enemy and boss entities further enrich the threat environment that underpins the game’s social deduction tension.

Chapter 5

Conclusions & Future Scope

Project Koschei successfully demonstrates the feasibility of deploying a memory-augmented Large Language Model as a real-time social deception agent within a multiplayer game environment. The system integrates a Unity-based first-person game client, a FastAPI backend, a ChromaDB vector memory store, and a locally hosted Ollama LLM into a cohesive architecture that enables the alien impostor to engage players through proximity-based chat, adapt its dialogue using stylometric mimicry, and maintain consistent deceptive narratives across multiple conversational rounds. The backend terminal results confirmed that the LLM pipeline correctly classified player messages, selected contextually appropriate deception strategies such as `seed_doubt`, and generated responses within sub-second latency under normal load. The implemented gameplay systems — including the dual-radius proximity grouping, conversation lifecycle management, NPC researcher interactions, and FSM-driven enemy and impostor behaviour — collectively produced an immersive and psychologically tense multiplayer experience consistent with the project’s design goals.

The project also establishes a meaningful contribution to the broader research areas of LLM-driven autonomous agents, real-time AI-human interaction, and social deception modelling. By replacing scripted impostor logic with a context-aware language model grounded in retrieved memory, Project Koschei moves beyond the fixed behavioural patterns of traditional social deduction games toward a system capable of open-ended, adaptive deception. The modular architecture ensures that individual components — the game client, session layer, AI backend, and memory system — can be independently developed, tested, and scaled, providing a robust foundation for continued research and feature development.

Future Scope

Several avenues exist for extending Project Koschei beyond its current implementation. The LLM backbone could be upgraded to larger models such as `llama3.3:70b` or fine-tuned on curated social deception dialogue datasets to produce more nuanced and harder-to-detect impostor behaviour. A dynamic trust graph system could be introduced to quantify inter-player suspicion levels in real time, allowing the impostor to prioritise accusation deflection toward the most suspicious player and enabling richer emergent social dynamics. Streaming inference via WebSockets or server-sent events could replace the current request–response model to reduce perceived response latency and make NPC dialogue feel more natural and conversational. The game world could be expanded with additional map zones, more NPC researcher personalities with distinct LLM-driven dialogue profiles, and secondary alien entity types with varied deception strategies to increase replayability. Finally, integrating text-to-speech synthesis for impostor and NPC responses, combined with voice-based proximity chat for human players, would substantially enhance immersion and open new research directions in multimodal social deception detection.

References

- [1] Barri, I., Roig, C., & Giné, F. (2016). Distributing game instances in a hybrid client-server/P2P system to support MMORPG playability. *Multimedia Tools and Applications*, 75(4), 2005–2029. <https://doi.org/10.1007/s11042-014-2370-5>
- [2] Chandrasekhara, A., et al. (2024). AMGPT: A large language model for contextual querying. *arXiv preprint arXiv:2406.00031*.
- [3] Chen, J., Lin, H., Han, X., & Sun, L. (2023). Benchmarking large language models in retrieval-augmented generation. Chinese Information Processing Laboratory, Institute of Software, Chinese Academy of Sciences; University of Chinese Academy of Sciences.
- [4] Danry, V., Pataranutaporn, P., Groh, M., & Epstein, Z. (2023). *Deceptive explanations by large language models lead people to change their beliefs about misinformation more often than honest explanations*. MIT Media Lab / Northwestern University / Stanford University.
- [5] Deng, Y., Li, Y., Seet, R., Tang, X., & Cai, W. (2018). The server allocation problem for session-based multiplayer cloud gaming. *IEEE Transactions on Multimedia*, 20(5), 1233–1245. <https://doi.org/10.1109/TMM.2017.2760622>
- [6] Hagendorff, T. (2023). Deception abilities emerged in large language models. [Preprint].
- [7] Hong, J., & He, L. (2025). Enhancing memory retrieval in generative agents through LLM-trained cross attention networks (ACAN). *Frontiers in Psychology*.
- [8] Kolenbrander, J., & Michaels, A. J. (2023). Personality emulation utilizing large language models.
- [9] Liu, H., et al. (2022). Relational memory augmented language models. *arXiv preprint arXiv:2201.09680*.

- [10] Mikros, G. (2023). Beyond the surface: Stylometric analysis of GPT-4o’s capacity for literary style imitation.
- [11] Motoo, T., Kawasaki, J., Fujihashi, T., Saruwatari, S., & Watanabe, T. (2021). Client-side network delay compensation for online shooting games. *IEEE Access*, 9, 125678–125692. <https://doi.org/10.1109/ACCESS.2021.3111343>
- [12] Muludi, K., Fitria, K. M., Triloka, J., & Sutedi, S. (2023). Retrieval-augmented generation approach: Document question answering using large language model. Darmajaya Informatics and Business Institute, Bandar Lampung, Indonesia.
- [13] Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. Stanford University & Google Research.
- [14] Park, P. S., Goldstein, S., O’Gara, A., Chen, M., & Hendrycks, D. (2023). AI deception: A survey of examples, risks, and potential solutions.
- [15] Poje, K., Brcic, M., Kovac, M., & Babac, M. B. (2023). Effect of private deliberation: Deception of large language models in game play.
- [16] Raith, L., Bignill, J., Stavropoulos, V., Millear, P., Allen, A., Stallman, H. M., Mason, J., De Regt, T., Wood, A., & Kannis-Dymand, L. (2021). Massively multiplayer online games and well-being: A systematic literature review. *Frontiers in Psychology*, 12, 698799. <https://doi.org/10.3389/fpsyg.2021.698799>
- [17] Wang, W., et al. (2024). LONGMEM: Augmenting large language models with long-term memory through decoupled memory retrieval and fusion. *arXiv preprint arXiv:2402.02244*.
- [18] Wang, Y., et al. (2024). Beyond the limits: Techniques to extend context length in large language models. *arXiv preprint arXiv:2402.02244*.
- [19] Yoo, B., & Kim, K.-J. (2023). Finding deceivers in social context with large language models and how to find them: The case of the Mafia game.
- [20] Zeng, Y., et al. (2023). Not just imitation: Enhancing role-playing through retrieve fine-tuned collaborative LLM

Appendix A: Presentation

Final Project Presentation

Project Koschei

A MULTIPLAYER NARRATIVE-DRIVEN GAME USING MEMORY AUGMENTED LLM FOR REAL-TIME SOCIAL DECEPTION

Guide: Mr. Sebin Jose

Serah Grace Kurian - U2203192
Vishwanath Pradeep - U2203212
Nandhaakishore S - U2203149
Sreenandan Pradeep - U2203199

March 6, 2026

Final Presentation

1

Contents

- Title slide
- Contents
- Problem Definition
- Project Objective
- Purpose and Need
- Literature Survey
- Proposed Method
- Architecture Diagram
- Module description
- Results and Outputs
- Assumptions
- Work breakdown and responsibilities
- Hardware and Software Requirements
- Gantt Chart
- Risks and Challenges
- Conclusion
- References

March 6, 2026

Final Presentation

2

Introduction

- **Concept:** A 3D third-person social deduction game blending psychological tension and tactical gameplay.
- **Setting:** A fictionalized Chernobyl Exclusion Zone, at a derelict Soviet research facility.
- **The Twist:** The team is infiltrated by a mutant impostor powered by a memory-augmented LLM.
- **Core Gameplay:** Focuses on unscripted communication, manipulation, and social reasoning to find the impostor.
- **Research Focus:** A platform to study:
 - LLM-driven agents in dynamic environments.
 - Real-time AI-human interaction and deception.
 - The effectiveness of long-term memory in AI dialogue.

March 6, 2026

Final Presentation

3

Problem Definition

LLMs often struggle to behave naturally in dynamic social settings, frequently forgetting previously established facts, lacking emotional adaptability, and failing to replicate individual communication styles in a natural and believable manner.

March 6, 2026

Final Presentation

4

Problem Objective

- To develop a 3D multiplayer story-driven social deduction game set in a Chernobyl-inspired environment with mission-based objectives, exploration, NPC assistance, and combat-based confrontations with the AI impostor.
- To integrate a locally hosted, memory-augmented LLM agent capable of deception, impersonation, emotional responses, and real-time adaptive dialogue.
- To implement AI-driven interactions including proximity-based chat, trust tracking, stylometric mimicry, and disguising mechanics.
- To design an NPC framework for environmental storytelling and player assistance.
- To build weapon mechanics, combat, and damage systems.

March 6, 2026

Final Presentation

5

Purpose and Need

- Addressing LLM limitations in dynamic social contexts: context retention, emotional adaptability, and style replication.
- Enabling tense, immersive gameplay through believable, unscripted AI-driven deception.
- Building a low-latency, locally hosted, memory-augmented LLM framework for multiplayer environments.
- Supporting research into trust dynamics, social deception, and adversarial AI-human interaction.
- Demonstrating broader applications in simulation training, storytelling, and conversational AI.
- Advancing techniques in stylometric mimicry, emotion-aware dialogue, and persistent agent memory.

March 6, 2026

Final Presentation

6

Literature Survey

Title: Personality Emulation Utilizing Large Language Models

Citation: Kolenbrander, J., & Michaels, A. J. (2025), Applied Sciences

Method:

- Applied Big Five personality model to generate email responses using GPT-4o with 33 personality vectors.
- Validated through seven LLMs and human surveys across professional, phishing, and romantic email contexts.

Advantages:

- LLMs can embed personality traits in text with 66.72% accuracy after normalization, validating stylometric mimicry feasibility.
- Big Five framework provides structured personality representation for consistent AI impostor personas.
- Multiple LLM validation shows personality detection is possible, supporting trust graph behavioral tracking.

Disadvantages:

- Limited to single responses, not extended conversations; our impostor requires conversation buffers for multi-turn consistency.
- 66.72% accuracy shows traits aren't always clearly perceived, affecting mimicry believability.
- Human perception only 46.48% accurate, requiring 20-message stylometry caching to capture nuanced linguistic patterns.

March 6, 2026

Final Presentation

7

Literature Survey

Title: Beyond the surface: Stylometric analysis of GPT-4o's capacity for literary style imitation

Citation: Mikros, G. (2025). Digital Scholarship in the Humanities, 40, 587-600.

Method:

Evaluated GPT-4o's literary style imitation using three prompting strategies (zero-shot, zero-shot imitation, in-context learning). Analyzed 45 texts with Random Forest classification, AMNP, word embeddings, LIWC features, and t-SNE visualization.

Advantages:

- LLMs capture surface stylistic features (83% accuracy), supporting stylometric mimicry.
- In-context learning (15k words) improves style alignment, supporting 20-message caching.
- AMNP features identify linguistic patterns for believable dialogue.

Disadvantages:

- GPT-4o struggles with deeper style elements (voice, emotion).
- Generated texts form separate clusters from originals.
- Large context (15k words) needed; 20-message limit may reduce fidelity.

March 6, 2026

Final Presentation

8

Literature Survey

Title: AMGPT: a Large Language Model for Contextual Querying in Additive Manufacturing

Citation: Achuth Chandrasekhar, Jonathan Chan, Francis Ogoke, Olabode Ajenifujah, Amir Barati Farimani, 2024, ScienceDirect

Method:

Developed AMGPT, a domain-specific LLM system using LLaMA2-7B combined with a Retrieval-Augmented Generation (RAG) pipeline. Additive manufacturing research papers were indexed using LlamaIndex and SentenceTransformer embeddings, allowing the model to retrieve relevant documents and generate context-aware answers.

Advantages:

- Provides domain-specific knowledge retrieval for additive manufacturing research.
- RAG reduces hallucinations by grounding responses in real documents.
- Uses a moderate-sized model (7B parameters), reducing training and computational cost.

Disadvantages:

- Accuracy depends on the quality and coverage of the document database.
- Requires GPU resources for efficient deployment.
- May generate incomplete answers if relevant documents are not retrieved.

March 6, 2026

Final Presentation

9

Literature Survey

Title: Finding deceivers in social context with large language models and how to find them: the case of the Mafia game
Citation: Yoo, B., & Kim, K. J. (2024). Scientific Reports.

Method:

- GPT-4 analyzes Mafia game transcripts using chain-of-thought reasoning to identify deceivers from conversational patterns, voting, and suspicion deflection—without fine-tuning.

Advantages:

- LLMs achieve 75% deception detection accuracy, validating their understanding of social deception for our AI impostor.
- Information accumulation boosts accuracy (33.33%→75.00%), supporting our ChromaDB memory approach.
- Provides explainable reasoning, informing our deception planning modules.

Disadvantages:

- Limited to observer role; our AI must actively engage and manipulate players in real-time.
- Weak early-game detection (33.33%), challenging our impostor's persona building.
- Requires substantial conversation data, necessitating our stylometry caching and buffers.

March 6, 2026

Final Presentation

10

Literature Survey

Title: A Survey and Taxonomy of Latency Compensation Techniques for Network Computer Games
Citation: Liu et al., ACM Computing Surveys (2022)

Methodology:

- Reviewed 80+ peer-reviewed papers and 20 industry blogs on network latency compensation.
- Created hierarchical taxonomy with four families: Feedback, Prediction, Time Manipulation, World Adjustment.

Advantages:

- Comprehensive framework maps relationships between latency compensation strategies.
- Validates industry practices like Extrapolation and Time Warp in commercial games.
- Provides clear implementation guidance on client vs. server placement.

Disadvantages:

- Reveals research gaps: over half lack user-based fairness evaluation.
- Genre bias toward FPS, neglecting MMORPGs and RTS games.
- No new algorithms—just classification of existing techniques.

March 6, 2026

Final Presentation

11

Proposed Method

Deceptive AI Backend (Python)

- ChromaDB: Three vector collections – player messages, game events, NPC memory – for semantic retrieval and session consistency
- Ollama LLM (Llama 3.2): Locally hosted; generates style-matched deceptive responses with rule validation and template fallback
- FastAPI Server: Orchestrates message ingestion, ChromaDB retrieval, deception strategy, and LLM prompting via REST API
- Stylometry Engine: Analyzes target player's last 10–20 messages to build a writing style profile for impersonation
- Deception Strategy: Temporal arc – gather info → build trust → seed doubt → accuse – with auto-switch to defense when accused
- Player Profiling: Extracts facts from messages – locations, actions, observations – stored as alibi material
- Impostor & NPC Modules: Manage agent behaviour, monster AI, and relay player context to the backend

March 6, 2026

Final Presentation

12

Proposed Method

Unity Gaming Services

- Lobby, Relay, and Authentication services manage multiplayer sessions
- Handle matchmaking, data transport, and player validation

Host/Server Side (Unity)

- Network Manager synchronizes game state across clients
- Game Manager tracks objectives, scores, and timers

Client Side (Unity)

- Player handles input, movement, and interactions
- UI Manager displays tasks and game information
- Syncs state with host via Network Manager

Data Flow

- Unity → Python Backend: Context and game state
- Python Backend → Unity: AI-generated impostor responses
- Host ↔ Clients: Synchronized multiplayer state

March 6, 2026

Final Presentation

13

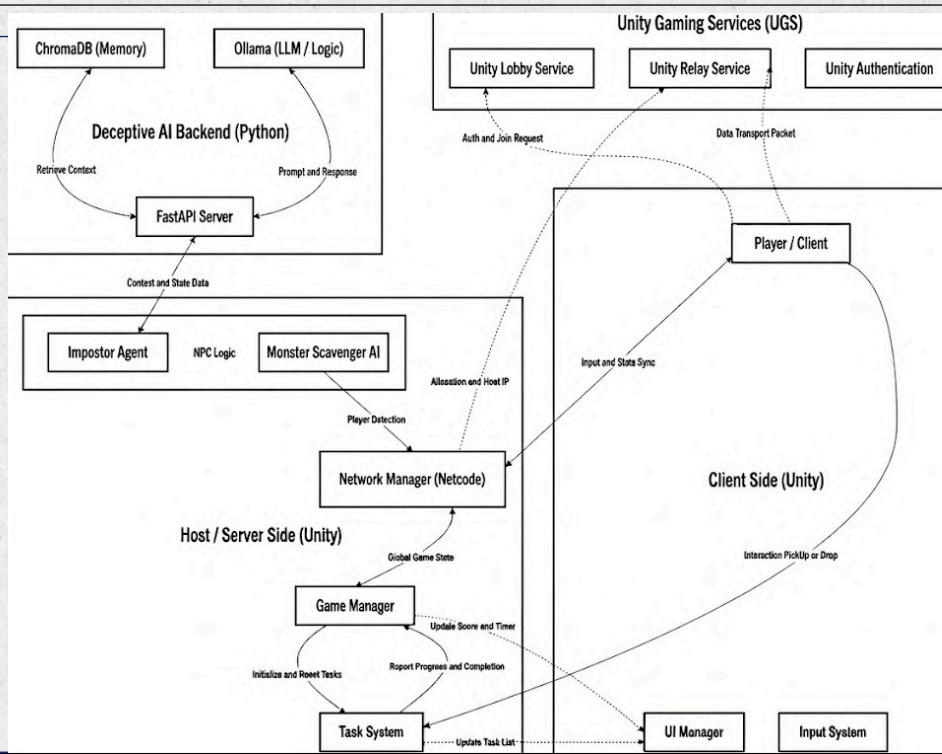
Design Diagrams

March 6, 2026

Final Presentation

14

Architecture diagram



March 6, 2026

15

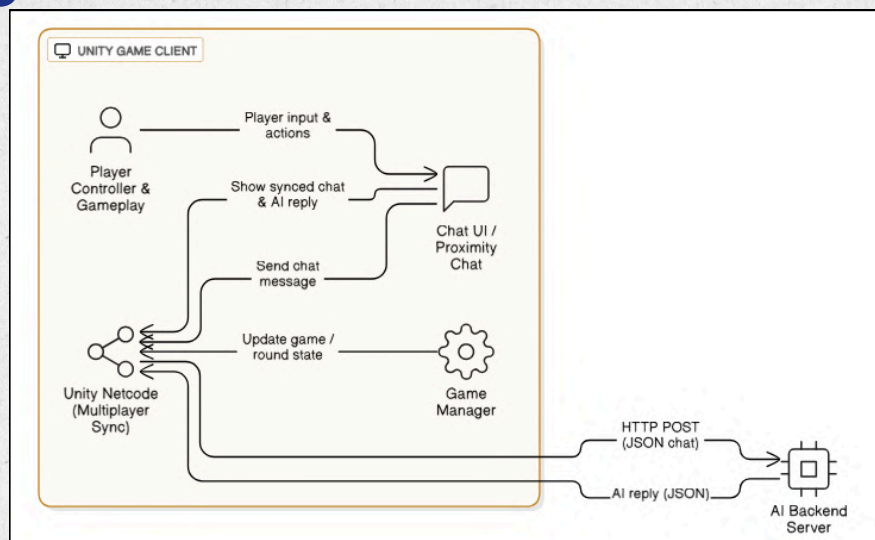
Module wise diagrams

March 6, 2026

Final Presentation

16

Player Chat Module

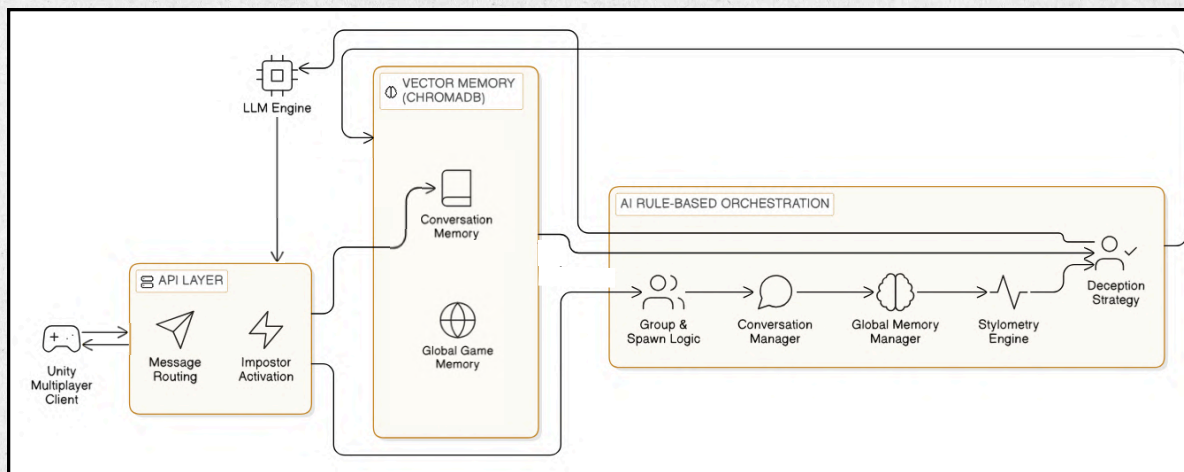


March 6, 2026

Final Presentation

17

AI Backend Module

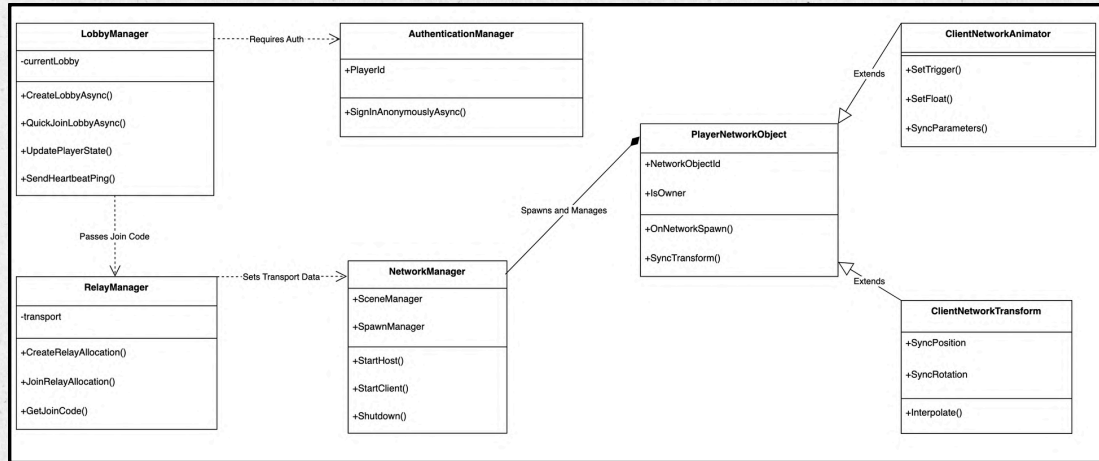


March 6, 2026

Final Presentation

18

Networking Module

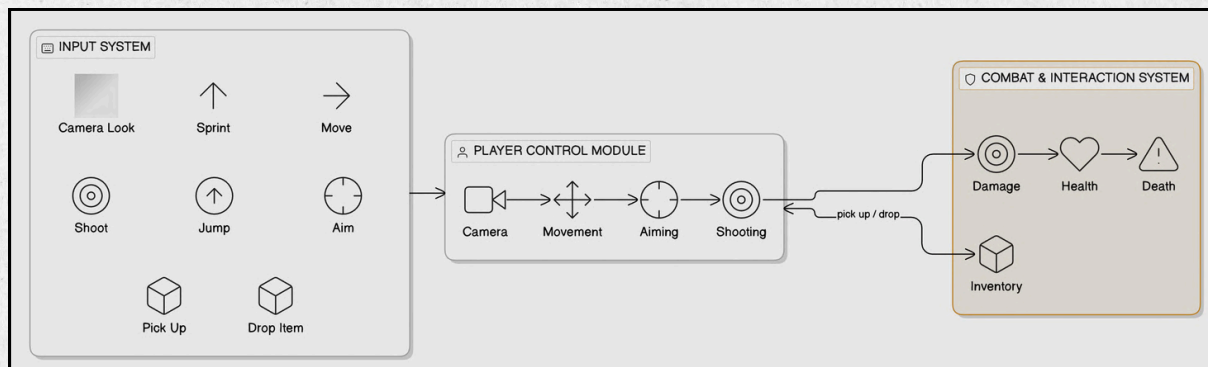


March 6, 2026

Final Presentation

19

Player control and Combat Module



March 6, 2026

Final Presentation

20

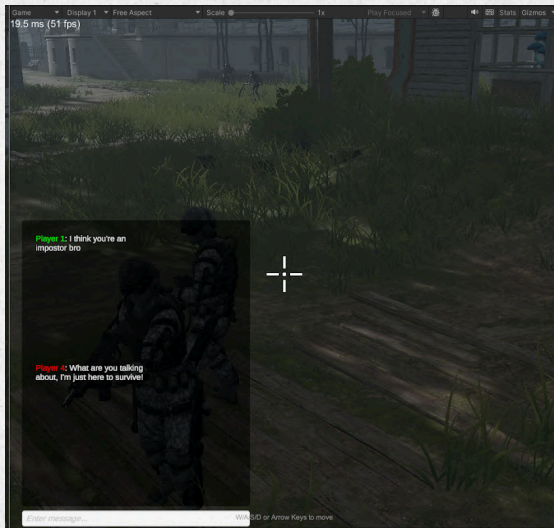
Results and Outputs

March 6, 2026

Final Presentation

21

Imposter Chat



```
=====
[Player 1] in group_Player 1: I think you're an imposter bro
Category: accusation
Respond? True (1-on-1 (100%))

=====
GENERATING STRATEGIC REPLY

Style: Casual gamer
Buffer: 7 messages
Facts: 1
Mode: seed_doubt
Strategy mode: seed_doubt (weak suspicions - seeding doubt)
Strategic: Not sure what's going on.....
0.57s
What are you talking about, I'm just here to survive!

=====
Replied as Player 4
=====
```

March 6, 2026

Final Presentation

22

Player UI



March 6, 2026

Final Presentation

23

Player damage



March 6, 2026

Final Presentation

24

Scientist NPC interaction



March 6, 2026

Final Presentation

25

Enemy with animations



March 6, 2026

Final Presentation

26

Impostor in game

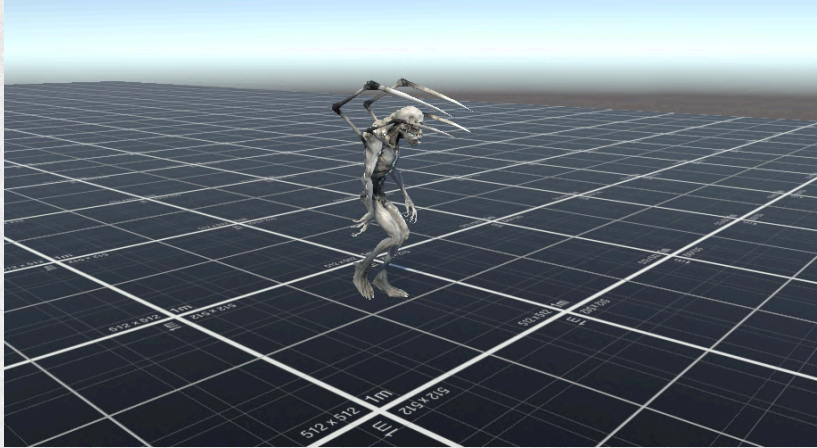


March 6, 2026

Final Presentation

27

Imposter (Unmasked) with animations Patrol



March 6, 2026

Final Presentation

28

Imposter (Unmasked) with animations Chase



March 6, 2026

Final Presentation

29

Imposter (Unmasked) with animations Attack



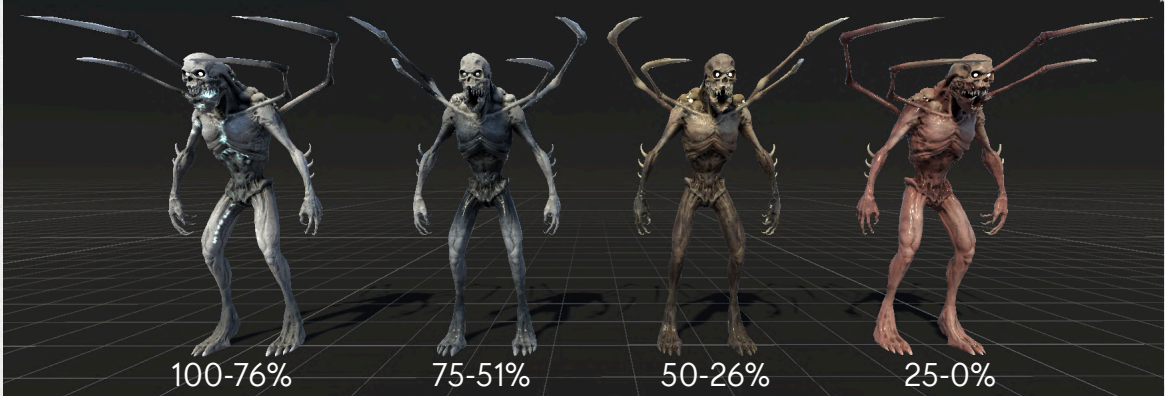
March 6, 2026

Final Presentation

30

*

Imposter (Unmasked) At diffrent health levels



March 6, 2026

Final Presentation

31

*

Imposter (Unmasked) with animations Dead



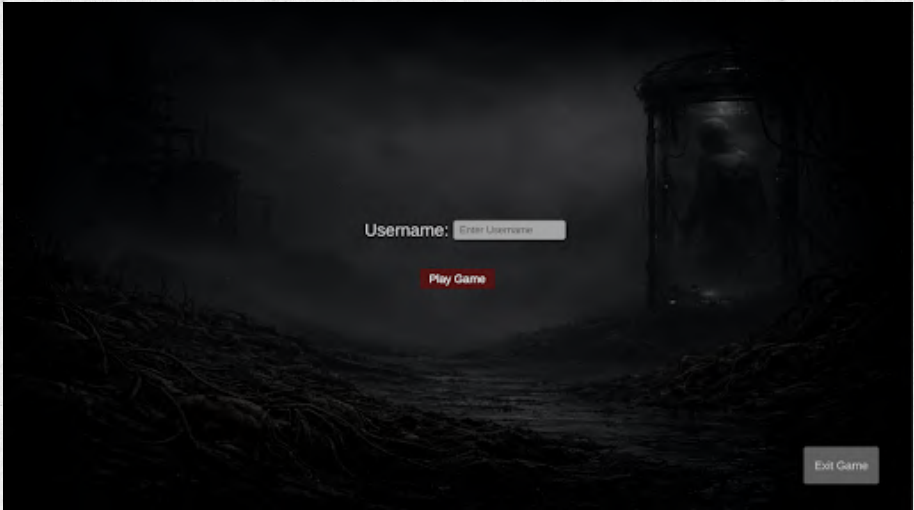
March 6, 2026

Final Presentation

32

*

Game Login

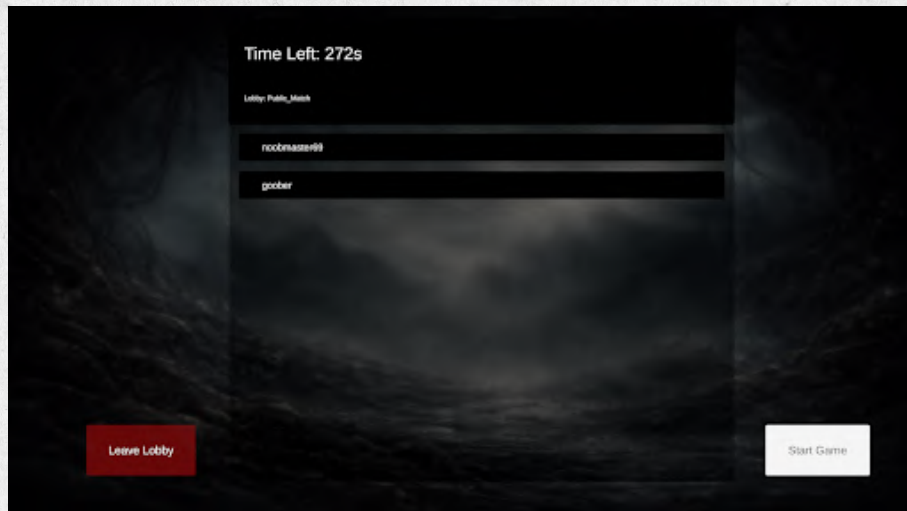


March 6, 2026

Final Presentation

33

Game Lobby



March 6, 2026

Final Presentation

34

Assumptions

- Players use proximity-based text chat, giving the AI impostor enough data to learn and mimic communication styles.
- The locally hosted LLM generates contextually appropriate deceptive responses with low latency for real-time gameplay.
- Unity Netcode synchronizes player movement, chat, weapons, NPC behavior, impostor state, and mission objectives across clients.
- Players have sufficient hardware to run the Unity client, multiplayer networking, and local LLM inference smoothly.
- The Chernobyl environment, NPC researchers, missions, and combat mechanics create an immersive social deduction experience.

March 6, 2026

Final Presentation

35

Roles

Primary Responsibilities	Team Member
Defines project vision, designs core mechanics, and oversees game balance.	Vishwanath
Implements the LLM cognitive core(locally), memory system, and Trust Graph in Python.	Serah
Implements player/alien abilities, NPCs, and core game mechanics.	Vishwanath and Serah
Manages multiplayer architecture.	Sreenandan
Creates all 3D assets, including environment, characters, and props and creates all sound effects, music, and NPC voice-overs..	Nandhaakishore
Rigs and animates characters, implements VFX, and optimizes performance.	Nandhaakishore
Develops and executes the test plan, manages bug tracking, and organizes playtesting.	Sreenandan

March 6, 2026

Final Presentation

36

Software Requirements

Component	Tool/Library
Game Engine	Unity 2022+ (URP or HDRP for 3D rendering)
Networking	Unity Netcode for GameObjects
Scripting	C#, Python 3.10+
LLM Hosting	Ollama or llama.cpp (for local inference)
Memory System	FAISS or ChromaDB (vector database)
LLM Backend API	FastAPI or Flask
Database	SQLite or lightweight JSON/NoSQL store (for player logs, trust, chat, etc.)

March 6, 2026

Final Presentation

37

Hardware Requirements

Type	Minimum	Recommended
CPU	Quad-core (e.g., Intel i5 / Ryzen 5)	6-core or better (e.g., i7 11 th gen / Ryzen 7)
RAM	8-16 GB	16-32 GB (especially for local LLM with memory)
Storage	256 GB SSD	512 GB+ SSD
GPU	Integrated or GTX 1650	RTX 2060-3060 +
Network	Stable LAN/Wi-Fi (for multiplayer)	Same

March 6, 2026

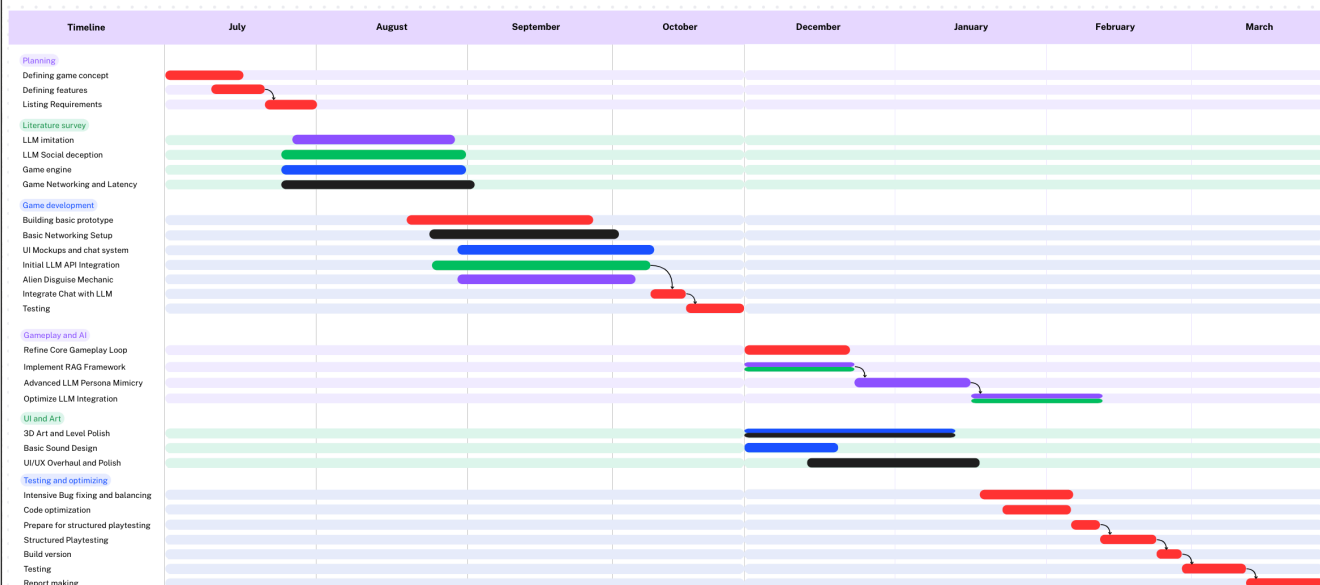
Final Presentation

38

Project Schedule

Project Timeline

Vishwanath Serah Nandhaakishore Sreenandan All



Risks and Challenges

- Performance Risk: AI processing could cause noticeable latency.
 - Mitigation: Optimize queries, use performant local LLMs, and asynchronous processing.
- Consistency Risk (Hallucination): AI may behave erratically or contradict itself.
 - Mitigation: Implement robust fallback logic and use strict prompt engineering.
- Immersion Risk: Poor mimicry could immediately expose the AI.
 - Mitigation: Refine the algorithm and focus on subtle mimicry over perfect replication.
- Synchronization Risk: Multiplayer state could de-sync due to LLM response delays.
 - Mitigation: Design a network tolerant of asynchronous AI responses.
- Ethical Risk: An AI designed for deception could create negative experiences.
 - Mitigation: Establish clear ethical boundaries and identify the AI post-game.

March 6, 2026

Final Presentation

40

Conclusion

Project Koschei advances interactive entertainment by creating a new type of antagonist: an autonomous, adaptive deceiver powered by a memory-augmented LLM.

Its key innovations—the Autonomous Deceiver, Dynamic Social Camouflage, and Emergent Narrative through Active Verification—solve long-standing issues of believability and replayability in game AI.

The project's impact extends beyond gaming, offering a valuable research platform for studying AI trust and alignment and providing a framework for applications in simulation training and interactive storytelling.

Project Koschei will set a new benchmark, compelling players to engage with a digital entity not based on what it's programmed to do, but on what it might be thinking.

March 6, 2026

Final Presentation

41

Thank You!

March 6, 2026

Final Presentation

42

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skills, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

PO1: Engineering Knowledge

Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis

Identify, formulate, review research literature and analyze complex engineering problems

reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions

Design creative solutions for complex engineering problems and design/develop systems/components/p to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems

Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions.

PO5: Engineering Tool Usage

Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World

Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics

Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work

Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication

Communicate effectively and inclusively within the engineering community and society at

large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance

Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning

Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Programme Specific Outcomes (PSO)

PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills

The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills

The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

SNO	DESCRIPTION	BLOOM'S TAXONOMY LEVEL
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant application.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

Appendix C: CO–PO–PSO Mapping

**COURSE OUTCOMES:MAPPING COURSE OUTCOMES (COs) –
PROGRAM OUTCOMES (POs) AND COURSE OUTCOMES (COs) –
PROGRAM SPECIFIC OUTCOMES (PSOs)**

	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3								3	2	2	1			3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2					1	3	3		
CO6			2			2	2	2	3	1	1			3

JUSTIFICATIONS FOR CO-PO MAPPING

MAPPING	LEVEL	JUSTIFICATION
101003/ CS822U.CO1- PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.
101003/ CS822U.CO1- PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1- PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1- PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1- PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1- PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1- PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1- PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1- PO10	Low	Involves basic planning and execution of tasks while implementing solutions.

MAPPING	LEVEL	JUSTIFICATION
101003/ CS822U.CO1- PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/ CS822U.CO1- PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.
101003/ CS822U.CO1- PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1- PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2- PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.
101003/ CS822U.CO2- PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2- PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2- PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2- PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2- PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.

MAPPING	LEVEL	JUSTIFICATION
101003/ CS822U.CO2- PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2- PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2- PO10	Low	Requires basic planning and coordination during development tasks.
101003/ CS822U.CO2- PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/ CS822U.CO2- PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2- PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/ CS822U.CO3- PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3- PO9	Medium	Requires effective communication among team members during collaboration.
101003/ CS822U.CO3- PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3- PO11	Low	Team interaction supports learning from peers and adapting new approaches.

MAPPING	LEVEL	JUSTIFICATION
101003/ CS822U.CO3- PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.
101003/ CS822U.CO4- PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4- PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4- PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4- PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4- PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/ CS822U.CO4- PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/ CS822U.CO4- PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4- PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/ CS822U.CO4- PO11	Low	Encourages adaptability and continuous learning during project implementation.

MAPPING	LEVEL	JUSTIFICATION
101003/ CS822U.CO4- PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/ CS822U.CO4- PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/ CS822U.CO5- PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5- PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5- PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5- PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5- PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5- PO10	Low	Requires minimal planning while proposing new solution approaches.
101003/ CS822U.CO5- PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/ CS822U.CO5- PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.

MAPPING	LEVEL	JUSTIFICATION
101003/ CS822U.CO6- PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6- PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6- PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6- PO8	Medium	Communication supports collaboration within team environments.
101003/ CS822U.CO6- PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6- PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6- PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6- PSO3	High	Communication of innovative computer science solutions and research outcomes.